



Modeling & Simulation

Calculus, Statistics, Probability,

2026 Spring

조상구교수

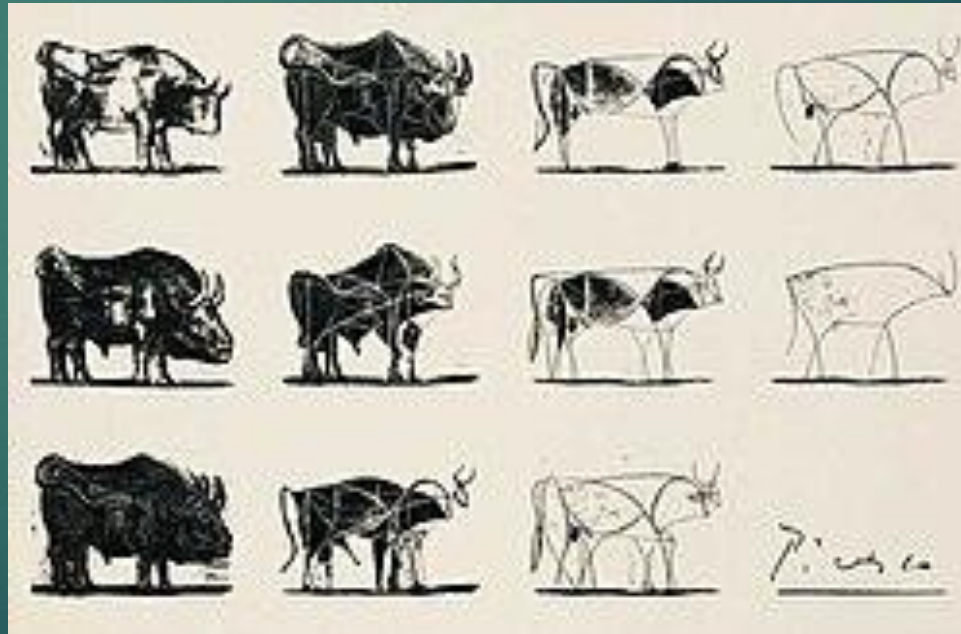
Contents

1. What is Modeling?
2. Python sympy
3. Time & Space (w1)
4. Optimization (w2)
5. Linear Regression (w3) (w4)
6. Optimization of loss function (L1, L2)
7. Short cut algorithm
8. Linear Programming
9. Transportation Problem
10. Montecarlo simulation
11. Random walk simulation
12. Bootstrap
13. Central limit theorem
14. Prince Frog
15. Month Hall
16. Genetic Algorithm
17. Differential Equation

Modeling

3

- 복잡한 현실 세계의 현상, 시스템, 사물 등을 일정한 규칙이나 관점에 따라 추상화하여 단순한 형태(모델)로 표현하는 과정(추상화, 단순화)
- 미래에 대한 결과를 논리적으로 추정하는 행위
 - 통계적 모델링
 - 머신러닝 모델링
 - 시뮬레이션 및 테스트
 - 수학적 모델링
 - 데이터베이스 모델링(ERD).....



- ❖ 불확실한 결과를 예측하기 위해 반복적인 무작위 추출을 사용하는 통계적 기법
- ❖ 결과가 확률에 의존하는 도박처럼, 수학적으로 계산하기 너무 복잡한 문제들을 수만 번의 가상 실험을 통해 그 근사값을 찾아내는 방식
- ❖ 몬테카를로의 작동 원리
 - ❖ 변수 정의: 결과에 영향을 주는 불확실한 요소들을 정의
 - ❖ 확률 분포 할당: 각 변수가 가질 수 있는 값의 범위와 확률(정규분포 등)을 설정
 - ❖ 반복 실험: 컴퓨터를 이용해 무작위 값을 입력하며 수천, 수만 번의 시뮬레이션을 실행
 - ❖ 결과 분석: 도출된 수많은 결과값의 평균이나 분포를 통해 최종 결과를 예측

Abstract + Object

5

복잡한 현실 세계는 시간에 따라 움직이고 있기 때문에 미분(derivative)을 이해하고 추상화와 모델링에 대한 이해를 하자(추상화는 OOP 구현)

단계	주요 활동	설명	구성 요소
추상화 (Abstraction)	특징, 기능 추출	현실 세계의 사람, 사물, 현상에서 불필요한 부분은 제거하고 서비스에 필요한 본질만 뽑아내는 과정	특징: 색상, 이름, 나이 기능: 달리기, 멈추기, 말하기
객체화 (Objectification)	속성, 메소드 정의	추상화된 개념을 프로그래밍 언어의 구조(Class)에 담아 실제 동작 가능한 단위로 만드는 과정	속성 (Variable): 데이터 저장 메소드 (Function): 로직 수행

Abstract + Object

6

대상	추상화 (특징 & 기능)	객체화 (속성 & 메소드)	코드
자동차 (사물)	특징: 모델명, 연료 타입 기능: 가속한다, 경적을 울린다	속성: model, fuel 메소드: accelerate(), honk()	<code>car.model = "Tesla"</code> <code>car.accelerate()</code>
은행원 (사람)	특징: 사원번호, 소속 지점 기능: 입금 처리, 상담	속성: emp_id, branch 메소드: deposit(), consult()	<code>staff.emp_id = 101</code> <code>staff.deposit()</code>
주문 (현상)	특징: 주문 번호, 결제 금액 기능: 주문 취소, 상태를 변경	속성: order_id, amount 메소드: cancel(), update_status()	<code>order.amount = 500</code> <code>order.cancel()</code>

OOP - python

7

```
class Account:
    def __init__(self, owner, balance=0):
        # 속성 (Attributes)
        self.owner = owner
        self.balance = balance

    # 메소드 (Methods)
    def deposit(self, amount):
        self.balance += amount
        return f"{amount}원 입금 완료. 잔액: {self.balance}원"

    def withdraw(self, amount):
        if amount > self.balance:
            return "잔액이 부족합니다."
        self.balance -= amount
        return f"{amount}원 출금 완료. 잔액: {self.balance}원"

# 객체화 (인스턴스 생성)
my_acc = Account("김철수", 1000)
print(my_acc.deposit(500)) # 500원 입금
print(my_acc.withdraw(2000)) # 잔액 부족 확인
```


Proto_OOP – java script

8

```
class Account {
  constructor(owner, balance = 0) {
    // 속성 (Attributes)
    this.owner = owner;
    this.balance = balance;
  }
  // 메소드 (Methods)
  deposit(amount) {
    this.balance += amount;
    return `${amount}원 입금 완료. 잔액: ${this.balance}원`;
  }
  withdraw(amount) {
    if (amount > this.balance) {
      return "잔액이 부족합니다.";
    }
    this.balance -= amount;
    return `${amount}원 출금 완료. 잔액: ${this.balance}원`;
  }
}

// 객체화 (인스턴스 생성)
const myAcc = new Account("이영희", 2000);
console.log(myAcc.deposit(1000)); // 1000원 입금
console.log(myAcc.withdraw(500)); // 500원 출금
```


Contents

1. What is Modeling?
2. Python sympy
3. Time & Space (w1)
4. Optimization (w2)
5. Linear Regression (w3) (w4)
6. Optimization of loss function (L1, L2)
7. Short cut algorithm
8. Linear Programming
9. Transportation Problem
10. Montecarlo simulation
11. Random walk simulation
12. Bootstrap
13. Central limit theorem
14. Prince Frog
15. Month Hall
16. Genetic Algorithm
17. Differential Equation

- ◆ sympy는 파이썬에서 사용 가능한 대수(algebra) 계산 라이브러리로, 기호 연산과 수학적 공식 표현을 지원
 - ◆ 기호 연산
 - ◆ 방정식 풀기
 - ◆ 미분과 적분
 - ◆ 방정식 그래프 그리기

python sympy

11

- ▶ `from sympy import symbols`
- ▶ `x, y = symbols('x y')`
- ▶ `expr = x + 2*y`

- ▶ `from sympy import diff, integrate`
- ▶ `expr = x**2 + 3*x + 2`
- ▶ `derivative = diff(expr, x)`
- ▶ `integral = integrate(expr, x)`

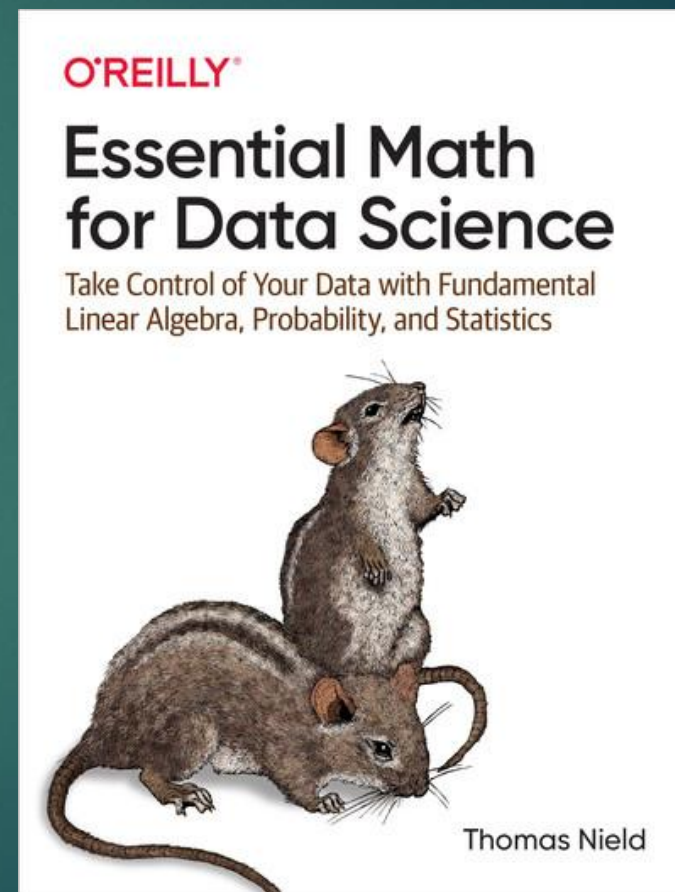
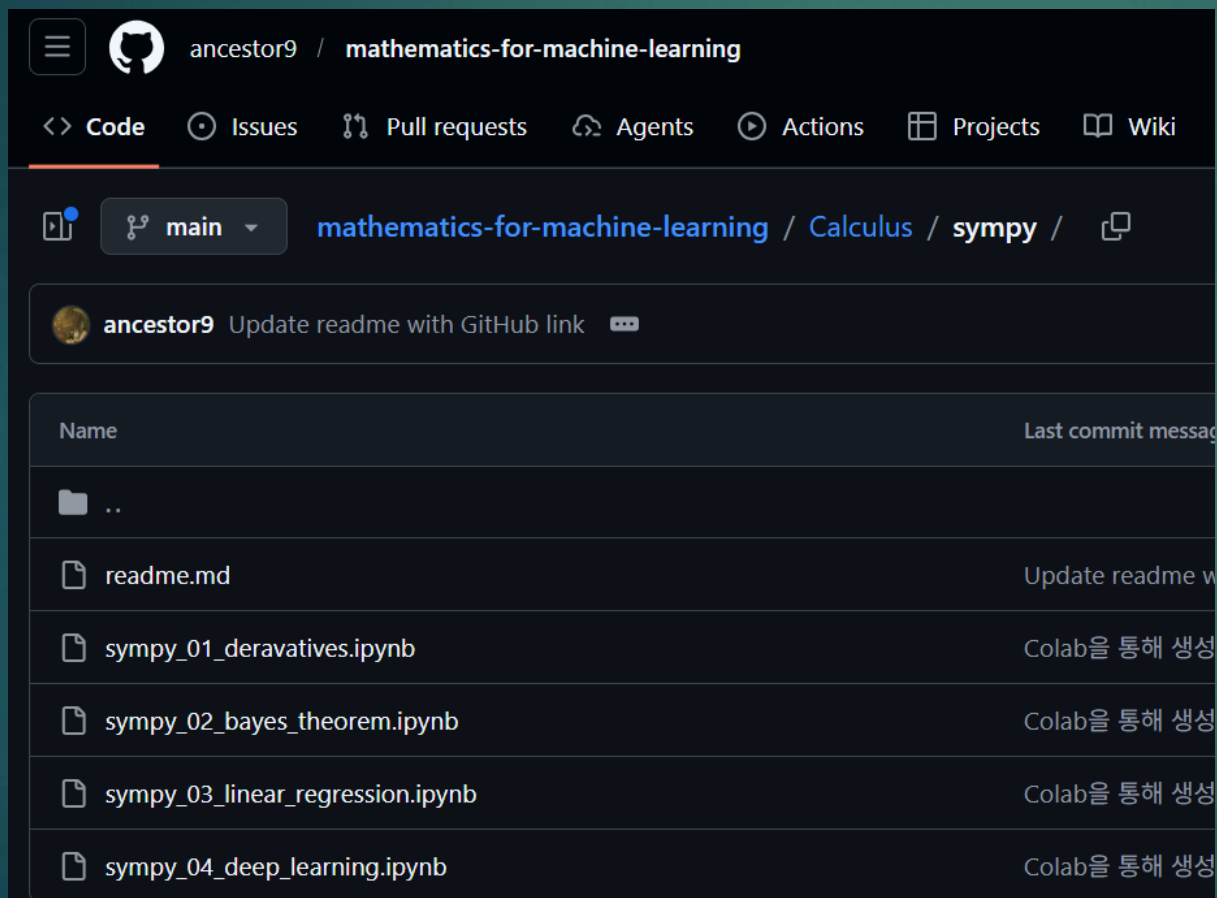
- ▶ `from sympy import Eq, solve`
- ▶ `eq = Eq(x**2 + 2*x + 1, 0)`
- ▶ `sol = solve(eq, x)`

- ▶ `from sympy.plotting import plot`
- ▶ `expr = x**2 - 3*x + 2`
- ▶ `plot(expr, (x, -5, 5))`
- ▶ `plot(diff(expr, x))`

python sympy

12

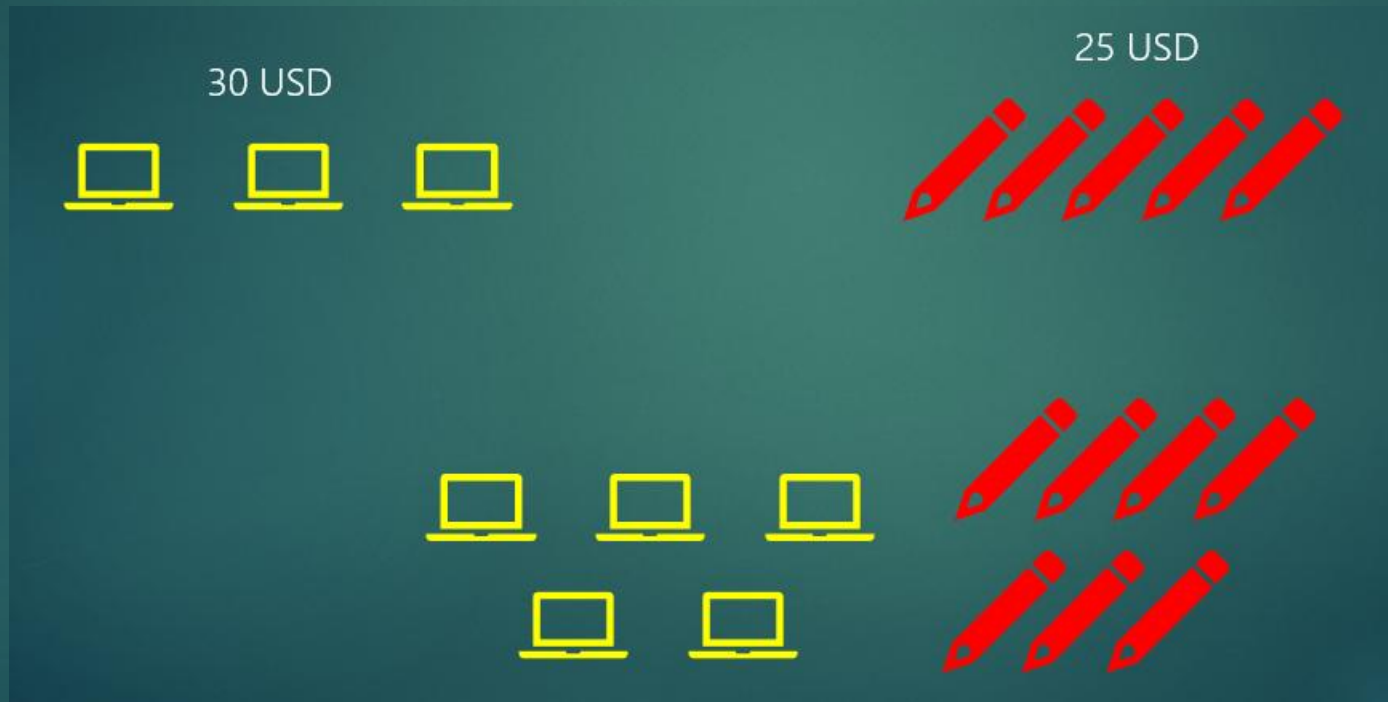
◆ 아래 4개 코드를 실습하기(recap)



partial derivative

13

- ◆ 조건 : 공책 3권에 30 USD, 펜 5자루에 25 USD 이다.
- ◆ 문제: 공책 5권과 펜 7자루를 사면 총 얼마일까?



partial derivative

14

- ◆ 조건 : 공책 1권당 10 USD, 펜 1자루에 5 USD
- ◆ 문제: 5 권 * 10\$ + 7 자루*5\$ = 85 \$
- ◆ 어떤 물건의 '단가'를 구해, 원하는 개수만큼 곱하는 기본 원리를 문자로 공식화

$$\text{Cost of } p \text{ objects} = \frac{\text{Cost of } n \text{ Objects}}{n} \times p$$

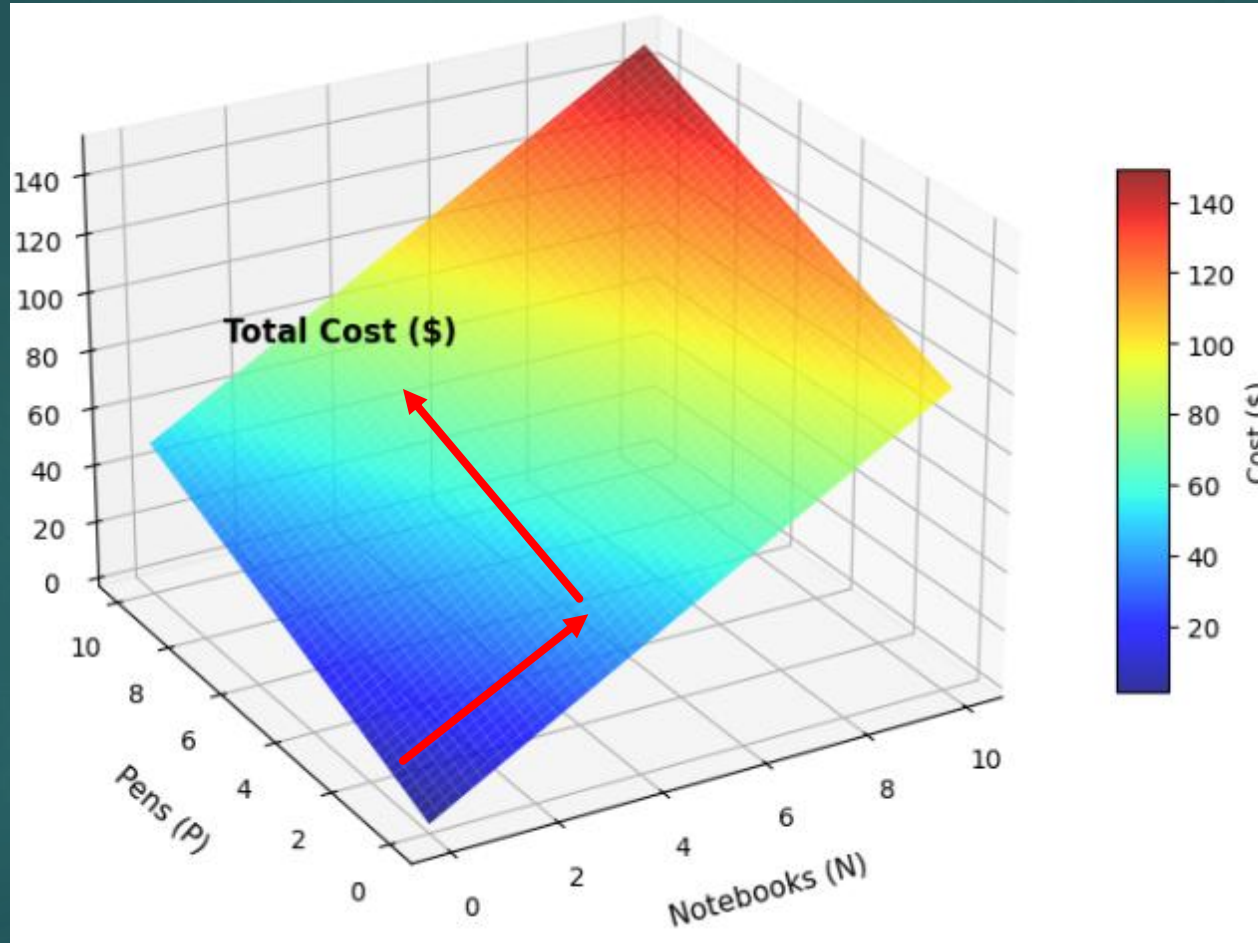
Cost of 3 Notebooks is 30 \$
Cost of 1 Notebook is 30/3 \$
Cost of 5 Notebooks is 30/3 × 5 \$
Which is 50 \$

Cost of 5 Pens is 25 \$
Cost of 1 Pen is 25/5 \$
Cost of 7 Pens is 25/5 × 7 \$
Which is 35 \$

$$\text{Cost}(N, P) = 10N + 5P$$

partial derivative

15



total, partial derivative

16

항목	공책 (Notebook) 의한 변화	펜 (Pen)에 의한 변화
미세 변화량 정의	공책 ∂N 개의 가격 = ∂C \$	펜 ∂P 자루의 가격 = ∂C \$
물건 1개당 단가 (편미분)	$\frac{\partial C}{\partial N}$ \$	$\frac{\partial C}{\partial P}$ \$
실제 추가 수량	dN 개	dP 자루
물건 추가 시 발생하는 비용	$\frac{\partial C}{\partial N} \times dN$ \$	$\frac{\partial C}{\partial P} \times dP$ \$

총 추가 비용은 = 공책 추가비용 + 펜 추가비용

$$dC = \left(\frac{\partial C}{\partial N} \times dN \right) + \left(\frac{\partial C}{\partial P} \times dP \right) \$$$

Jacobian Matrix

17

공책의 개수 변화(dN)와 펜의 개수 변화(dP)가 시스템에 입력되면, 자코비안(Jacobian Matrix) 행렬을 거쳐 세 가지 결과, 가격 변화(dC), 무게 변화(dW), 부피 변화(dV)로 출력

$$\begin{bmatrix} dC \\ dW \\ dV \end{bmatrix} = \begin{bmatrix} \frac{\partial C}{\partial N} & \frac{\partial C}{\partial P} \\ \frac{\partial W}{\partial N} & \frac{\partial W}{\partial P} \\ \frac{\partial V}{\partial N} & \frac{\partial V}{\partial P} \end{bmatrix} \begin{bmatrix} dN \\ dP \end{bmatrix}$$

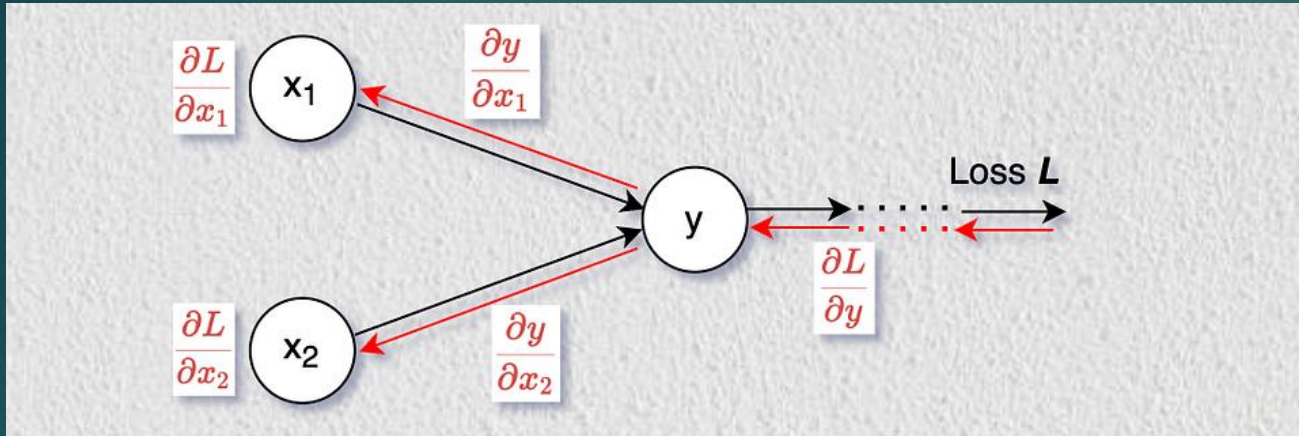
1행 (가격 스펙): $\begin{bmatrix} \frac{\partial C}{\partial N} & \frac{\partial C}{\partial P} \end{bmatrix} = [\text{공책 1권당 가격} \quad \text{펜 1자루당 가격}]$
2행 (무게 스펙): $\begin{bmatrix} \frac{\partial W}{\partial N} & \frac{\partial W}{\partial P} \end{bmatrix} = [\text{공책 1권당 무게} \quad \text{펜 1자루당 무게}]$
3행 (부피 스펙): $\begin{bmatrix} \frac{\partial V}{\partial N} & \frac{\partial V}{\partial P} \end{bmatrix} = [\text{공책 1권당 부피} \quad \text{펜 1자루당 부피}]$

자코비안(Jacobian Matrix)

back propagation, jacobian matrix

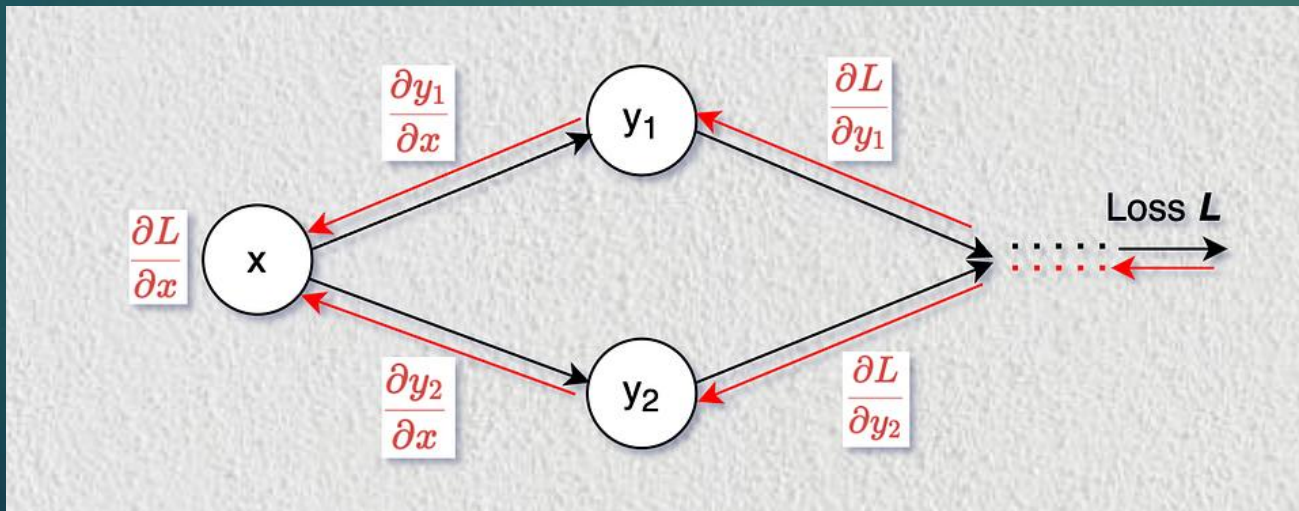
18

backpropagation-with-vectors-in-neural-network



$$\frac{\partial L}{\partial x_1} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial x_1}$$

$$\frac{\partial L}{\partial x_2} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial x_2}$$

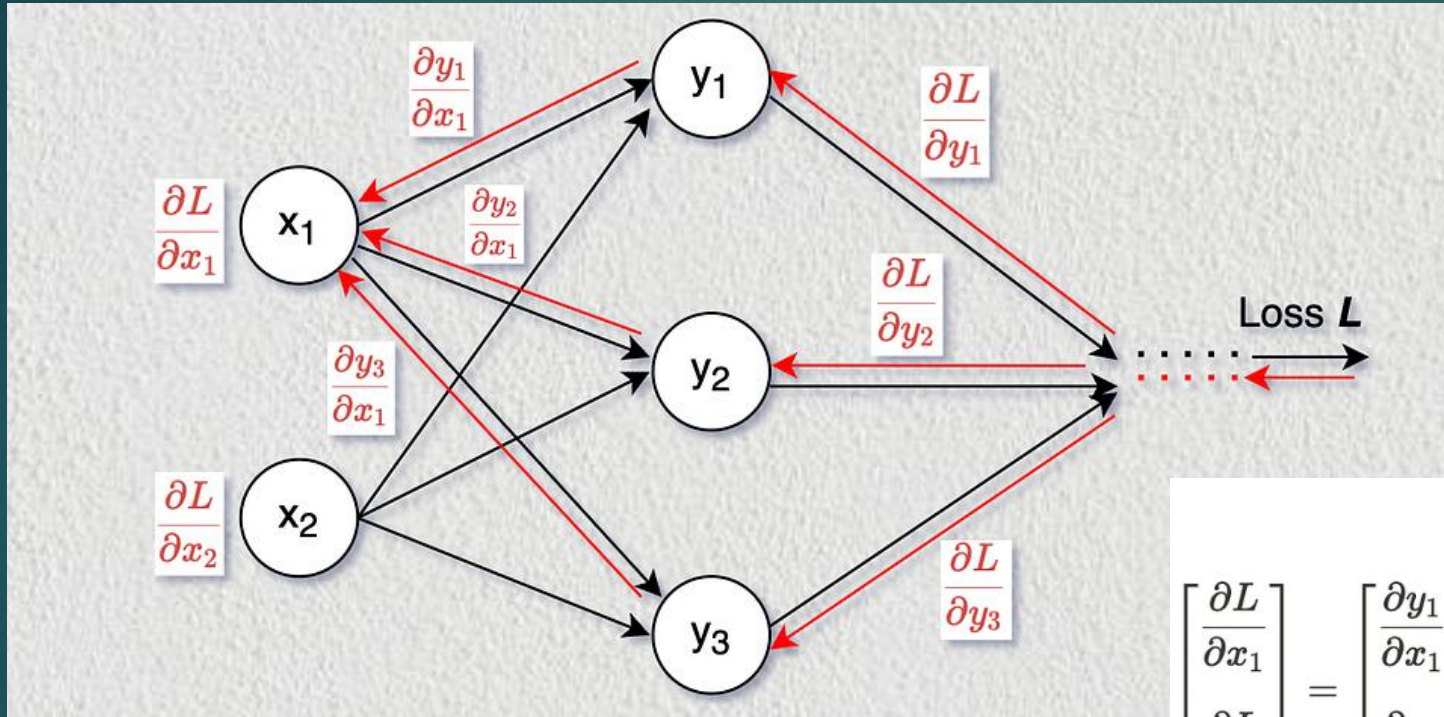


$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y_1} \frac{\partial y_1}{\partial x} + \frac{\partial L}{\partial y_2} \frac{\partial y_2}{\partial x}$$

back propagation, jacobian matrix

19

backpropagation-with-vectors-in-neural-network



$$\frac{\partial L}{\partial x_1} = \frac{\partial L}{\partial y_1} \frac{\partial y_1}{\partial x_1} + \frac{\partial L}{\partial y_2} \frac{\partial y_2}{\partial x_1} + \frac{\partial L}{\partial y_3} \frac{\partial y_3}{\partial x_1}$$
$$\frac{\partial L}{\partial x_2} = \frac{\partial L}{\partial y_1} \frac{\partial y_1}{\partial x_2} + \frac{\partial L}{\partial y_2} \frac{\partial y_2}{\partial x_2} + \frac{\partial L}{\partial y_3} \frac{\partial y_3}{\partial x_2}$$

$$\begin{bmatrix} \frac{\partial L}{\partial x_1} \\ \frac{\partial L}{\partial x_2} \end{bmatrix} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_2}{\partial x_1} & \frac{\partial y_3}{\partial x_1} \\ \frac{\partial y_1}{\partial x_2} & \frac{\partial y_2}{\partial x_2} & \frac{\partial y_3}{\partial x_2} \end{bmatrix} \begin{bmatrix} \frac{\partial L}{\partial y_1} \\ \frac{\partial L}{\partial y_2} \\ \frac{\partial L}{\partial y_3} \end{bmatrix} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} \\ \frac{\partial y_3}{\partial x_1} & \frac{\partial y_3}{\partial x_2} \end{bmatrix}^T \begin{bmatrix} \frac{\partial L}{\partial y_1} \\ \frac{\partial L}{\partial y_2} \\ \frac{\partial L}{\partial y_3} \end{bmatrix}$$

back propagation, jacobian matrix

20

backpropagation-with-vectors-in-neural-network

$$\begin{bmatrix} \frac{\partial L}{\partial x_1} \\ \frac{\partial L}{\partial x_2} \end{bmatrix} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_2}{\partial x_1} & \frac{\partial y_3}{\partial x_1} \\ \frac{\partial y_1}{\partial x_2} & \frac{\partial y_2}{\partial x_2} & \frac{\partial y_3}{\partial x_2} \end{bmatrix} \begin{bmatrix} \frac{\partial L}{\partial y_1} \\ \frac{\partial L}{\partial y_2} \\ \frac{\partial L}{\partial y_3} \end{bmatrix}$$

Backprop chain rule in matrix form for a layer with **column vector** input and output

$$\begin{bmatrix} \frac{\partial L}{\partial x_1} & \frac{\partial L}{\partial x_2} \end{bmatrix} = \begin{bmatrix} \frac{\partial L}{\partial y_1} & \frac{\partial L}{\partial y_2} & \frac{\partial L}{\partial y_3} \end{bmatrix} \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} \\ \frac{\partial y_3}{\partial x_1} & \frac{\partial y_3}{\partial x_2} \end{bmatrix}$$

Backprop chain rule in matrix form for a layer with **row vector** input and output

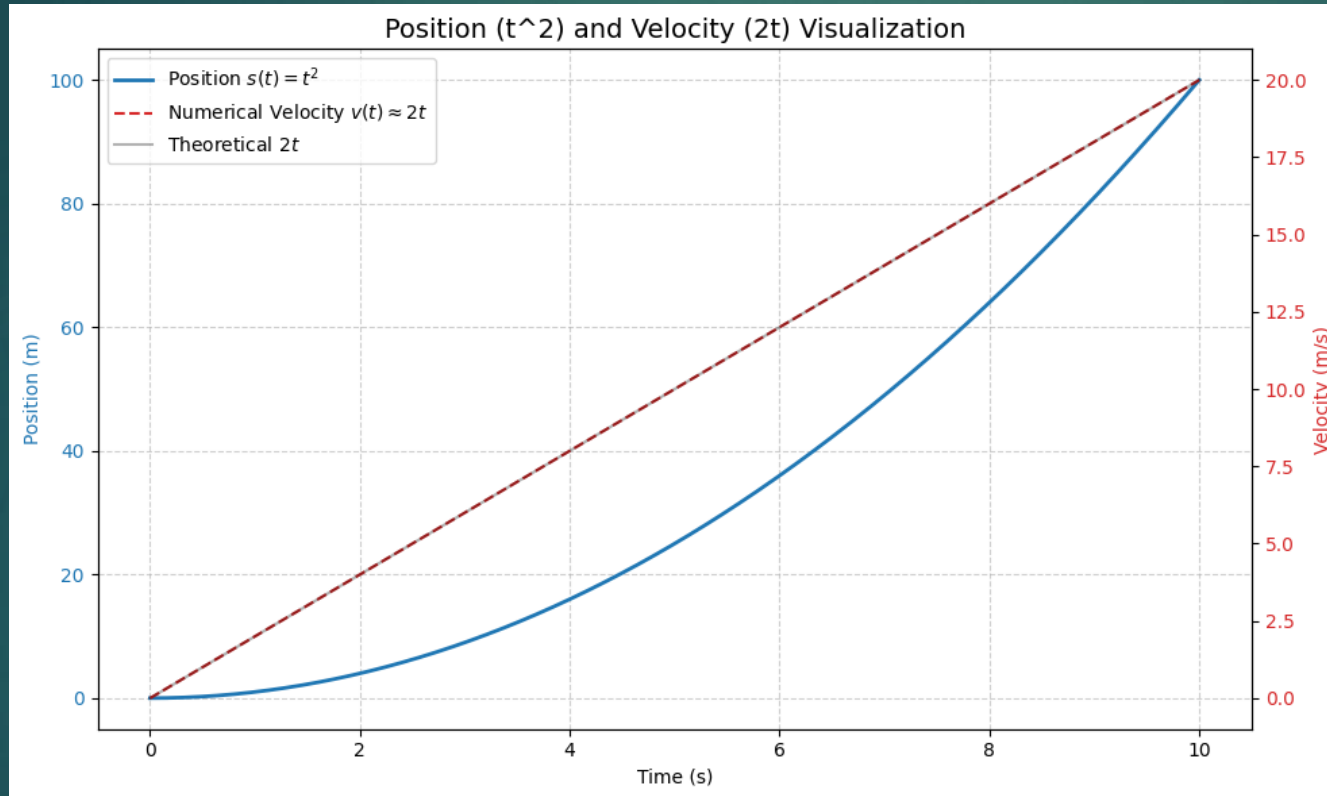
Contents

1. What is Modeling?
2. Python sympy
3. Time & Space (w1)
4. Optimization (w2)
5. Linear Regression (w3) (w4)
6. Optimization of loss function (L1, L2)
7. Short cut algorithm
8. Linear Programming
9. Transportation Problem
10. Montecarlo simulation
11. Random walk simulation
12. Bootstrap
13. Central limit theorem
14. Prince Frog
15. Month Hall
16. Genetic Algorithm
17. Differential Equation

Time & Space

22

- ▶ 위치함수 $s(t) = t^2$ 에 대한 순간 속도를 중앙차분법과 numpy 함수를 이용하여 구해보자



$$v(t) \approx \frac{s(t + \Delta t) - s(t - \Delta t)}{2\Delta t}$$

```
np.gradient(s, dt)
```

Time & Space

23

$$v(t) \approx \frac{s(t + \Delta t) - s(t - \Delta t)}{2\Delta t}$$

```
import numpy as np

def central_difference(f, x, eps=1e-6):
    """
    중앙 차분법을 이용한 미분값 계산
    f: 미분할 함수,      x: 미분값을 구할 지점
    eps: 아주 작은 변화량 (epsilon)
    """
    # 공식:  $v(t) \approx [s(t + \Delta t) - s(t - \Delta t)] / (2 * \Delta t)$ 
    numerator = f(x + eps) - f(x - eps)
    denominator = 2 * eps
    return numerator / denominator

def target_function(x):
    return x**2

# 설정값
x_point = 5; epsilon = 0.000001

# 실행
result = central_difference(target_function, x_point, epsilon)
print(f"지점 x = {x_point} 에서의 미분값: {result}")
print(f"실제 이론값 (2x): {2 * x_point}")
print(f"오차: {abs(result - 2 * x_point)}")
```

np.gradient(s, dt)

```
import numpy as np
import matplotlib.pyplot as plt

# 1. 데이터 생성 (시간 t와 위치 s)
# 0초부터 10초까지 0.1초 간격 (총 101개 지점)
t = np.linspace(0, 10, 101)
dt = t[1] - t[0] # 시간 간격 (delta t)
# 위치 함수 s(t) = t^2 (이론적 속도 v(t) = 2t)
s = t**2
velocity = np.gradient(s, dt)

# 3. 결과 확인 (이론값과 비교)
theoretical_v = 2 * t

print(f"시간 5초에서 수치적 속도: {velocity[50]:.2f}")
print(f"시간 5초에서 이론적 속도: {theoretical_v[50]:.2f}")
```

Time & Space

24

시속 10km로 등속도로 달리는 자동차로 3시간을 주행하면 이동한 전체 거리(km)를 구하는 파이썬 코드를 제시하라. (가속도, 감속도 경우도 이해하자)

```
import sympy as sp

# 위치 함수
s = 10 * t

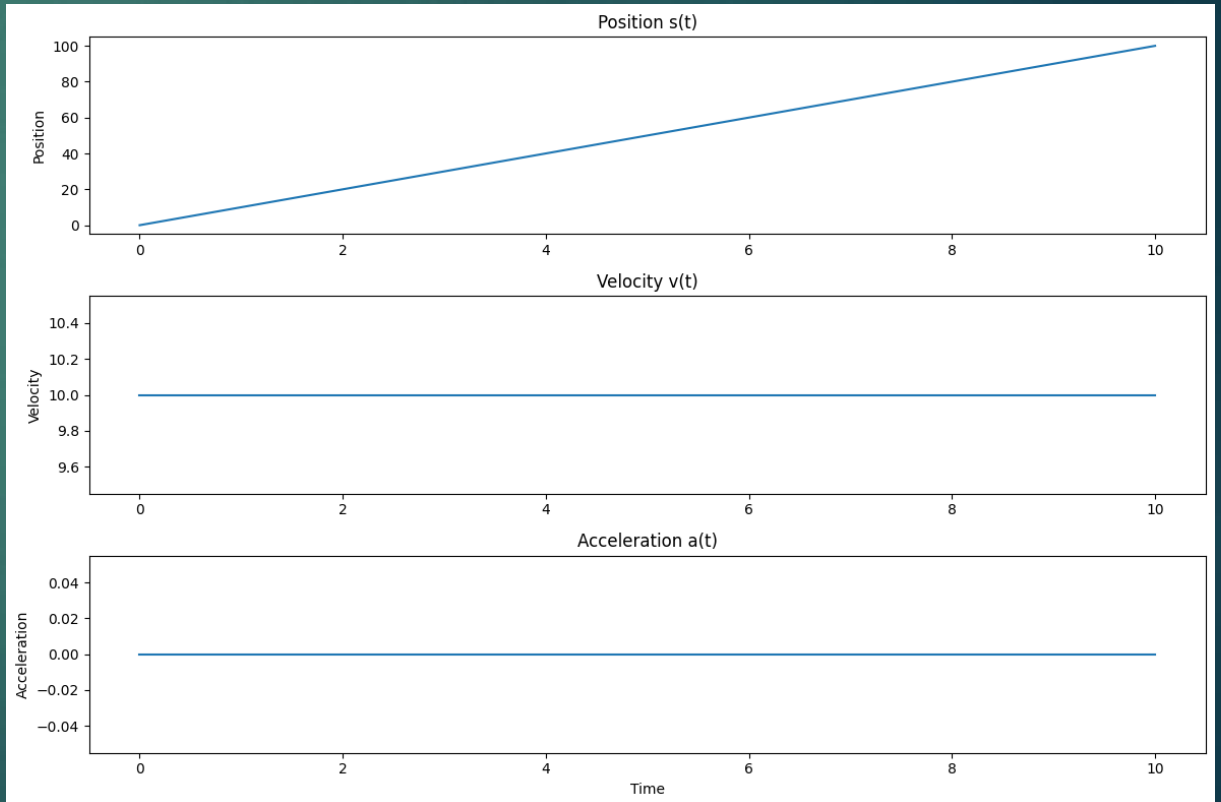
# 속도 함수
v = sp.diff(s, t)

# 가속도 함수
a = sp.diff(v, t)

s, v, a
```

등속도(미분방정식)

$$\frac{ds}{dt} = 10$$



Time & Space

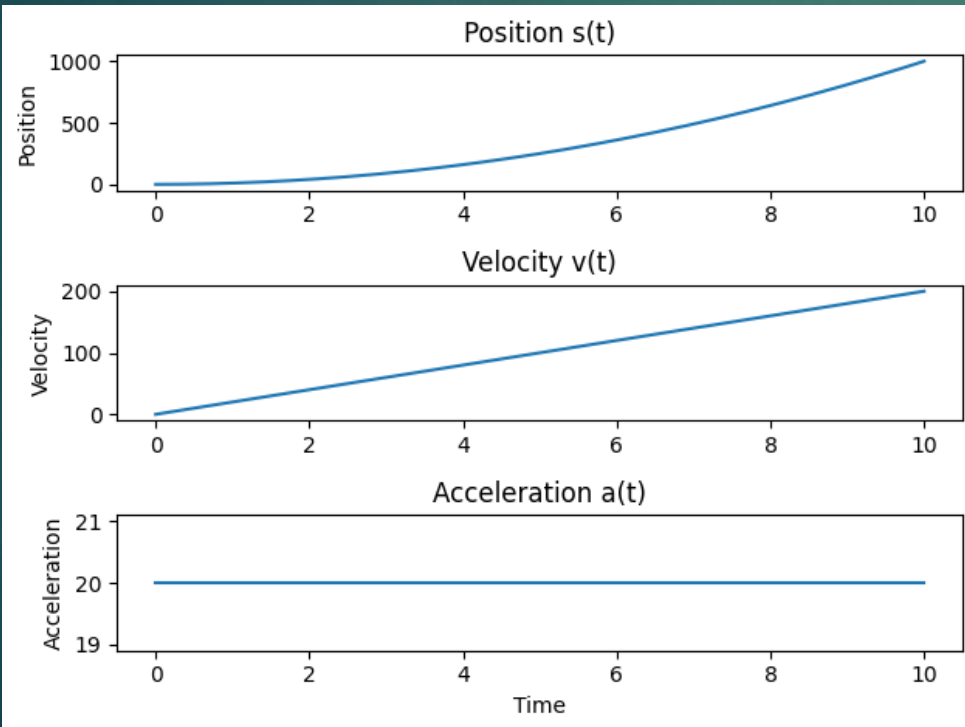
25

가속과 감속

가속도(미분방정식)

위치 함수
 $s = 10 * t^2$

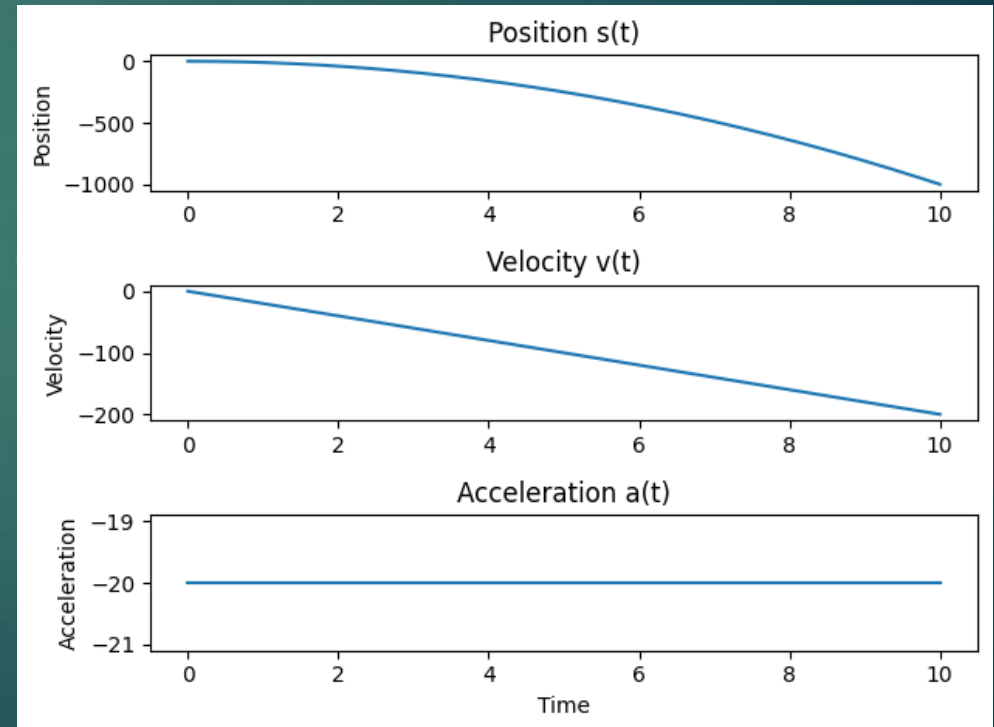
$$\frac{ds}{dt} = 10t$$



감속도(미분방정식)

위치 함수
 $s = -10 * t^2$

$$\frac{ds}{dt} = -10t$$



Time & Space

26

- 위치를 미분 $\rightarrow$ 속도, 속도를 미분 $\rightarrow$ 가속도
- 가속도를 적분 $\rightarrow$ 속도 속도를 적분 $\rightarrow$ 위치

물리량	의미	수학
위치	어디 있는가	$s(t)$
속도	얼마나 빨리 움직이는가	$\frac{ds}{dt}$
가속도	속도가 얼마나 변하는가	$\frac{dv}{dt}$

$$a(t) \rightarrow \int \rightarrow v(t) \rightarrow \int \rightarrow s(t)$$

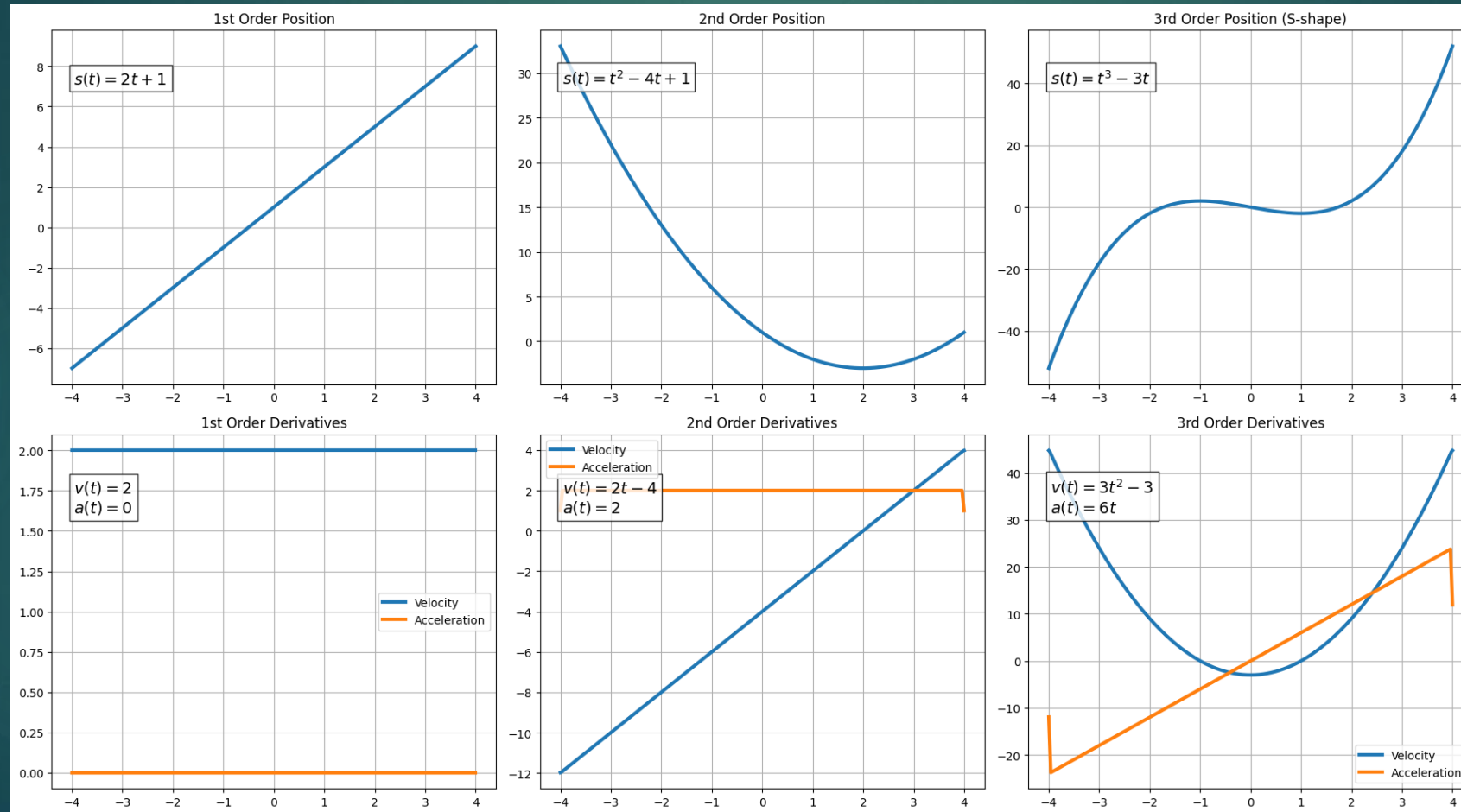
$$s(t) \rightarrow \frac{d}{dt} \rightarrow v(t) \rightarrow \frac{d}{dt} \rightarrow a(t)$$

- ▶ 미분과 적분은 단순히 복잡한 수식이 아니라, 움직이는 변화를 관찰하는 도구
 - ▶ 미분 (Differentiation)
 - ▶ 위치를 시간에 대해 미분하면 속도가 되고, 속도를 미분하면 가속도 : 순간적인 변화율
 - ▶ 적분 (Integration)
 - ▶ 가속도를 시간에 대해 적분하면 속도가 되고, 속도를 적분하면 변화된 거리(위치) : 누적된 총량

Time & Space

28

- ▶ 시간(t)에 따른 위치(s), 속도(v), 가속도(a)의 관계는 미적분을 이해하는 물리적 모델



$$v(t) = \frac{ds}{dt} = s'(t)$$

$$a(t) = \frac{dv}{dt} = v'(t) = \frac{d^2s}{dt^2} = s''(t)$$

- ▶ 당신은 서울에서 강릉까지(180 km) 여행을 가는데, 처음 90 km는 교통체증으로 90분이 걸렸고 뒤 90 km는 도로가 뽕 뚫려 45분 만에 달렸다.
 - ▶ 전체 평균속도 v_{av} 를 km/h 단위로 계산하세요.
 - ▶ 각 구간의 속도는 각각 60 km/h와 120 km/h입니다.
 - ▶ 단순히 두 속도의 산술평균(90 km/h)과 왜 다를까요? “총 거리 ÷ 총 시간”이라는 평균속도의 정의를 이용해 설명하시오

Time & Space

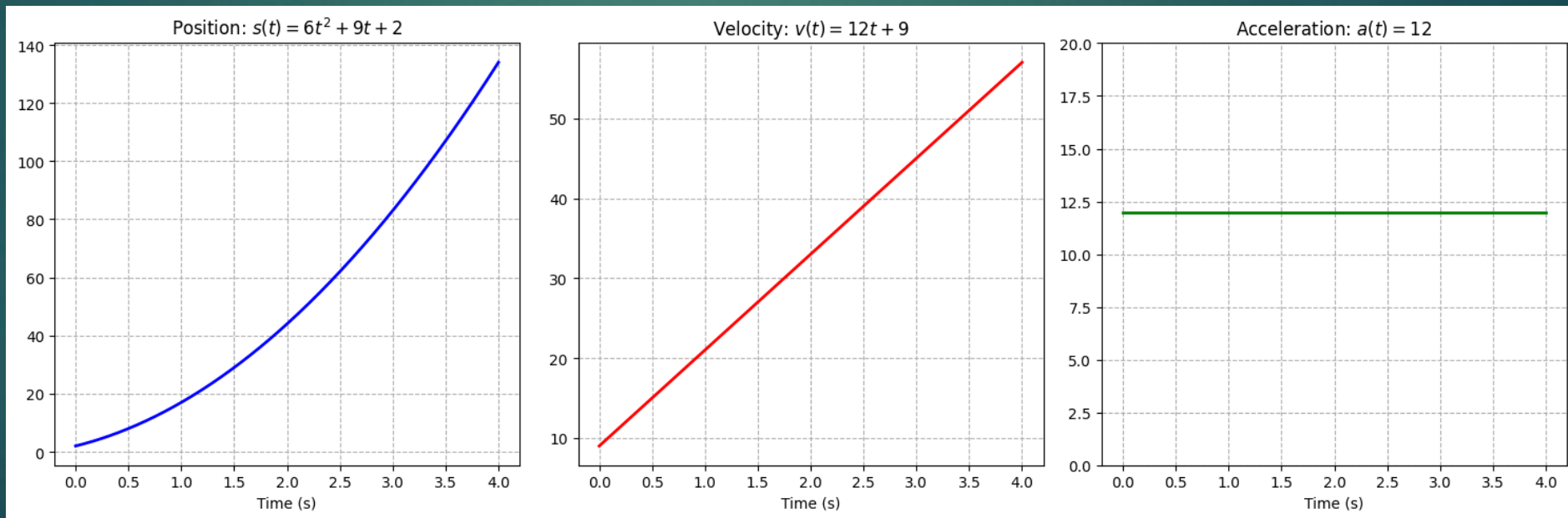
30

- ▶ 당신은 마블의 순간 이동 능력으로 당신의 위치가 시간에 따라 아래와 같이 주어졌어요.
 - ▶ $x(t) = 4t^2 + 2t$ (단위: m, t: 초)
- ▶ $t=0$ 초부터 $t=5$ 초까지의 평균속도 $v_{av} = \frac{\Delta x}{\Delta t}$ 를 구하시오
- ▶ 순간속도 함수 $v(t) = \frac{dx}{dt}$ 를 구하고, $t=3$ 초에서의 순간속도를 계산하시오
- ▶ 평균속도와 순간속도를 설명하시오

Time & Space

31

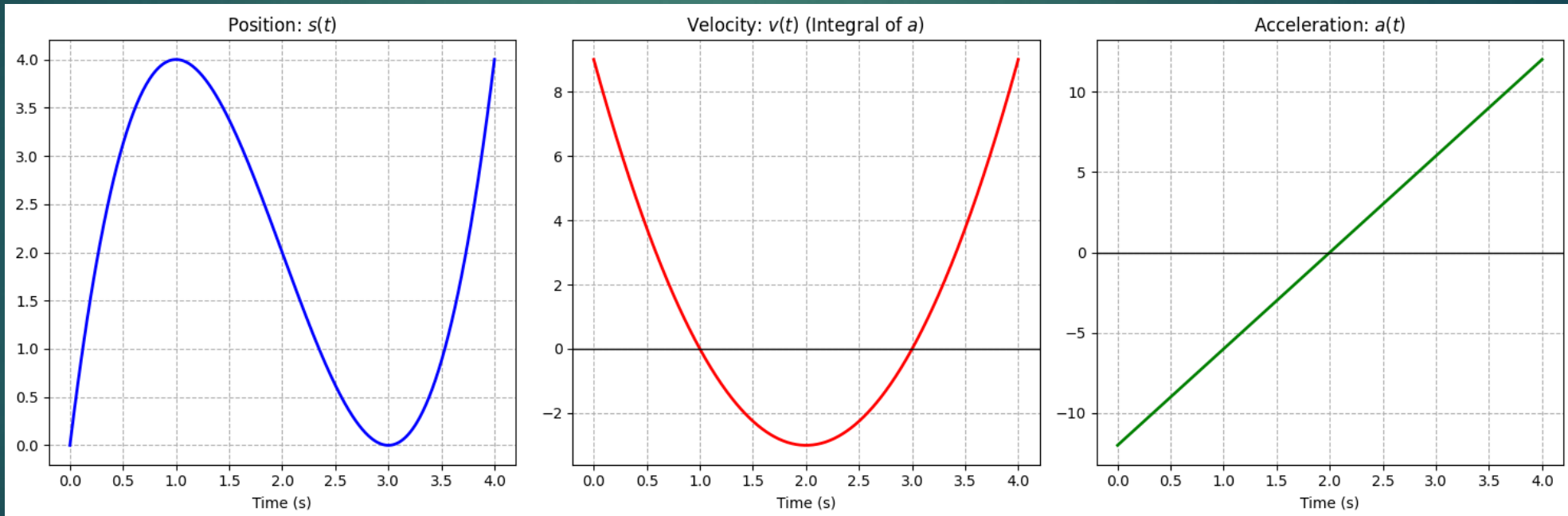
- ▶ 마블의 순간 이동에 따른 위치(변화된 거리)함수가 $s(t) = 6t^2 + 9t$ 인 어떤 물체의 움직임을 살펴보자
 - ▶ 시간변화 $\rightarrow$ 거리변화
 - ▶ 시간변화 $\rightarrow$ 속도 변화
 - ▶ 특정시간에 위치는? ($t = 2$)일 때 나의 위치(움직인 거리)는?



Time & Space

32

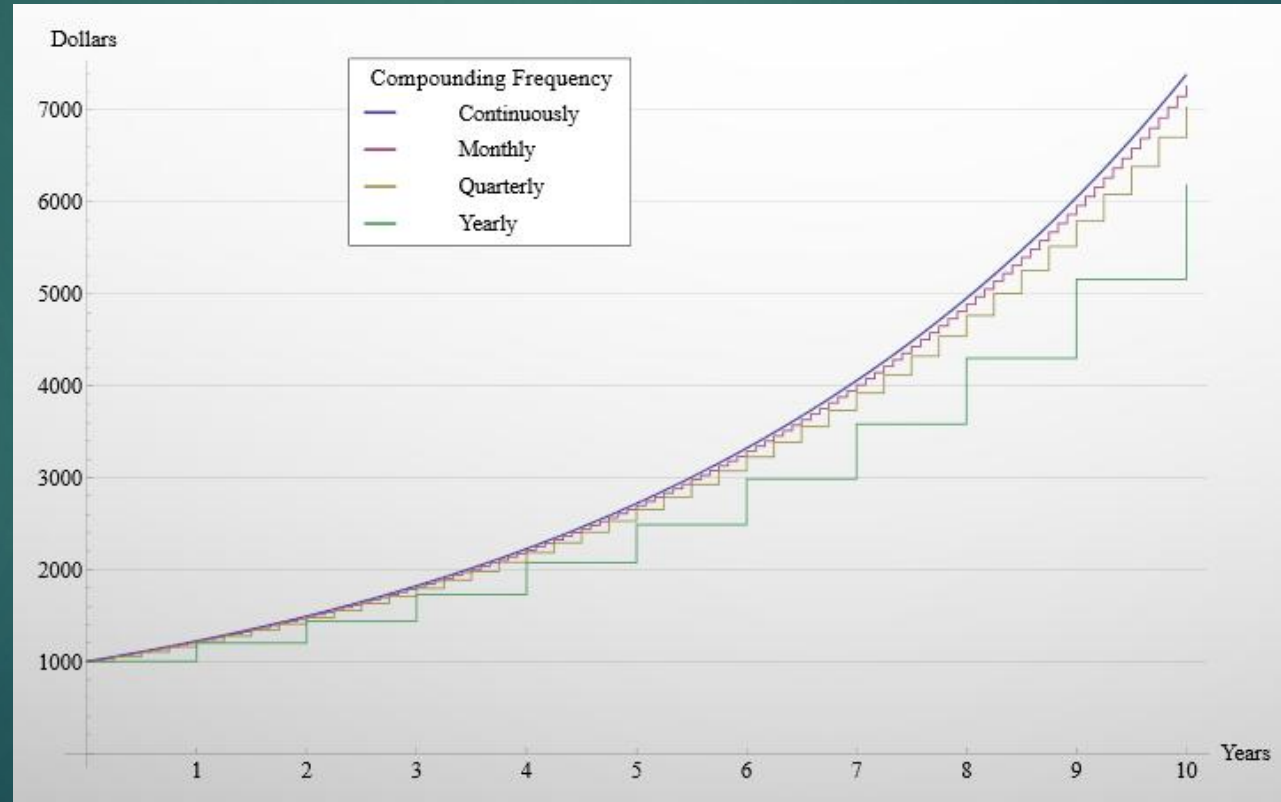
- ▶ 위치(변화된 거리)함수가 $s(t) = t^3 - 6t^2 + 9t$ 인 어떤 물체의 움직임을 살펴보자
 - ▶ 시간변화 $\rightarrow$ 거리변화
 - ▶ 시간변화 $\rightarrow$ 속도 변화
 - ▶ 특정시간에 위치는? ($t = 2$)일 때 나의 위치(움직인 거리)는?



Time & Space

33

- ▶ 아래 그림은 1,000 US dollars를 은행에 복리이자(compound interest)로 월복리, 1분기복리, 년 복리 그리고 시간을 무한히 나누어 복리이자를 한 경우의 그래프이다.
 - ▶ 미분방정식과 미래 가치 등을 계산하여 시각화를 재현하라



Contents

1. What is Modeling?
2. Python sympy
3. Time & Space (w1)
4. Optimization (w2)
5. Linear Regression (w3) (w4)
6. Optimization of loss function (L1, L2)
7. Short cut algorithm
8. Linear Programming
9. Transportation Problem
10. Montecarlo simulation
11. Random walk simulation
12. Bootstrap
13. Central limit theorem
14. Prince Frog
15. Month Hall
16. Genetic Algorithm
17. Differential Equation

▶ A, B 2군데 공급업체의 물품 가격 추세를 바탕으로 최적 조달 비율을 결정하라

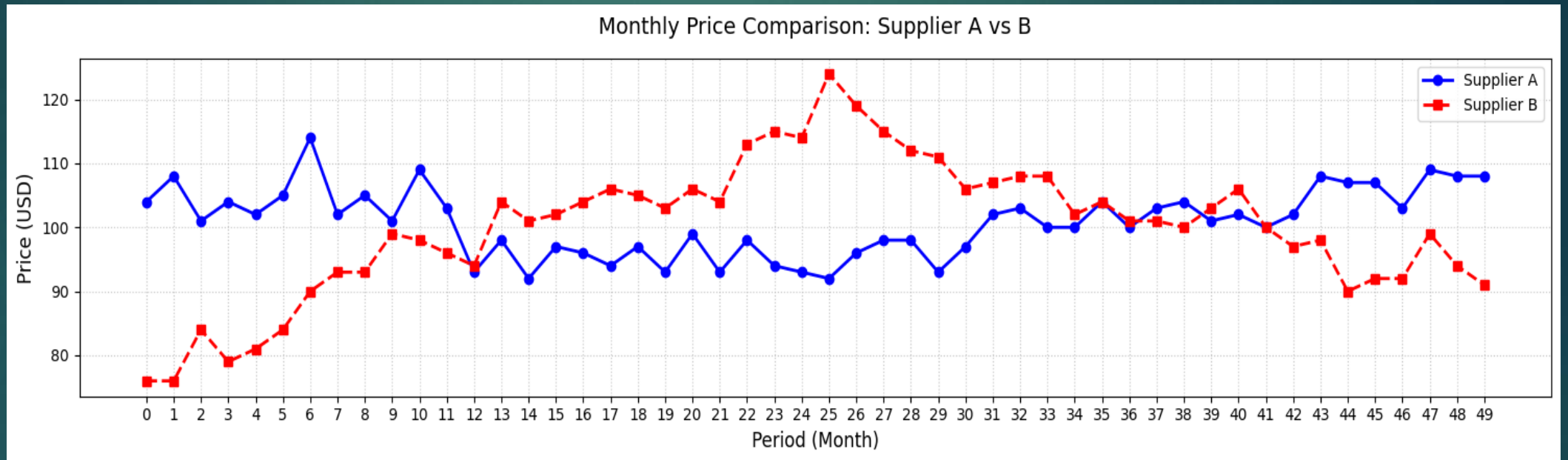
▶ 배경설명 (데이터는 Obtain_data_prices.csv)

- ❖ 제품 P (생산 공정의 필수 원자재)기간: 2018년 2월 ~ 2020년 3월 (과거 26개월 데이터 기반)
- ❖ 목적: 향후 12개월 예산 수립을 위한 공급업체(A, B) 간 최적 구매 비율 결정
- ❖ 데이터 분석 시나리오 (예상 분석 프로세스) : 공급업체 선정 및 비율 결정을 위해 다음 세 가지 지표를 검토하라
 - 1) 산술 평균 가격 비교 (기대 비용)두 업체 중 어느 곳이 전반적으로 더 저렴한가?만약 라면, 비용 절감 측면에서 A가 유리하지만 단순히 평균만 보서는 안 되며, 특정 시점의 가격 역전(Price Cross) 빈도를 확인하라
 - 2) 변동성 및 리스크 분석 (표준편차)가격이 얼마나 널뛰는가?예산 모델링에서 안정성은 매우 중요하기때문에 평균 가격이 낮더라도 변동성이 너무 크면 예산 초과 리스크가 발생
 - 3) 상관관계 분석 (포트폴리오 효과)두 업체의 가격이 같이 움직이는가?상관계수 확인: 두 업체의 가격 상관관계가 낮거나 음수라면, 물량을 나누어 구매함으로써 전체 조달 비용의 변동성을 상쇄(Hedge)할 수 있음

Optimization

36

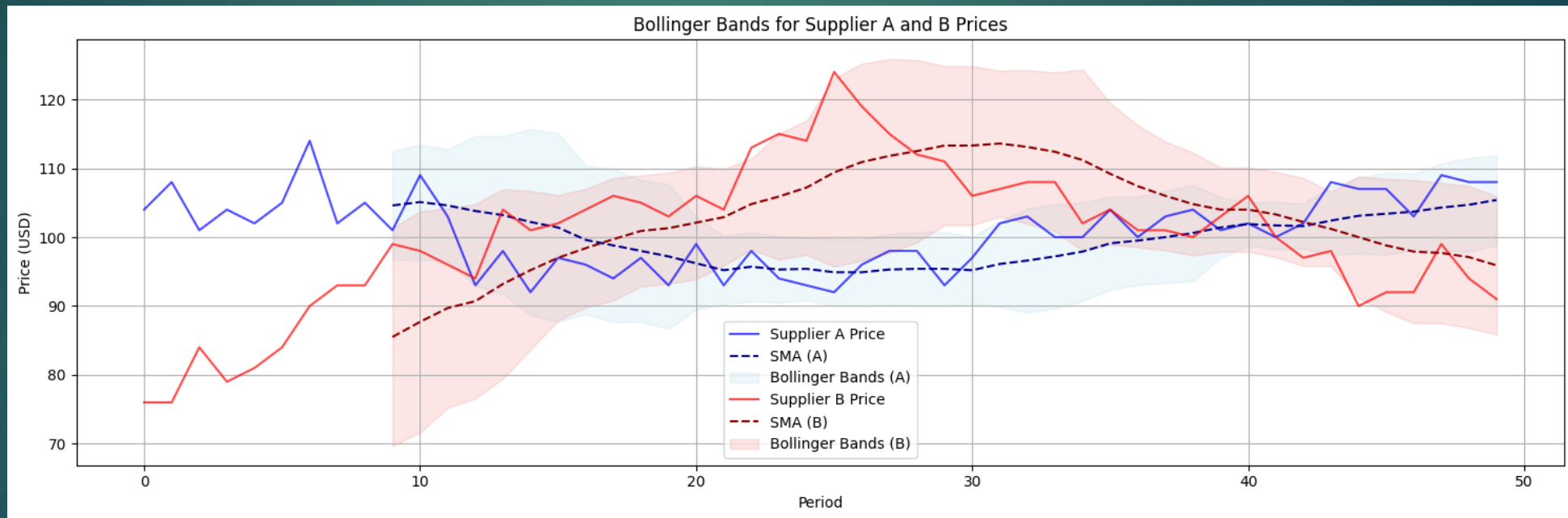
- ▶ A, B 2군데 공급업체의 물품 가격 추세를 바탕으로 최적의 주문 비율을 정하라



Optimization

37

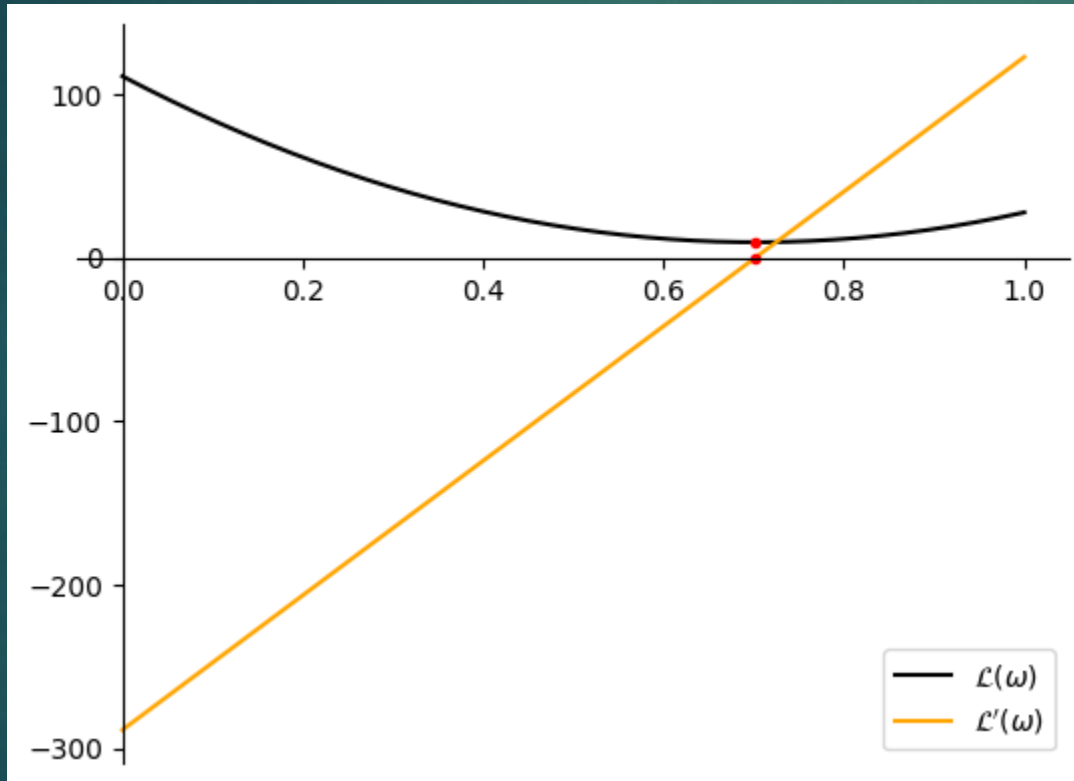
- ▶ A, B 2군데 공급업체의 물품 가격의 이동평균과 이동 표준편차를 고려하라
- ▶ (Hints) 가격의 변동성(분산)이 큰 것은 위험한 것 (Cost Minimization)



Optimization

38

- ▶ sympy로 비용함수를 만들어 최적화(미분 값이 zero)가 되는 w 를 찾으면 0.7023



```
import sympy as sp
```

```
# 1. 기호 정의
```

```
# w: 배분 비율, sA: A의 분산, sB: B의 분산, sAB: A와 B의 공분산
```

```
w = sp.symbols('omega')
```

```
sA, sB, sAB = sp.symbols('sigma_A^2 sigma_B^2 sigma_AB')
```

```
# 2. 리스크 함수(분산) 정의
```

```
#  $L(w) = \text{Var}(pA*w + pB*(1-w))$ 
```

```
# 전개식:  $w^2*\text{Var}(A) + (1-w)^2*\text{Var}(B) + 2*w*(1-w)*\text{Cov}(A,B)$ 
```

```
L = w**2 * sA + (1-w)**2 * sB + 2*w*(1-w) * sAB
```

```
# 3. 최적화를 위한 미분 ( $dL/dw = 0$ )
```

```
diff_L = sp.diff(L, w)
```

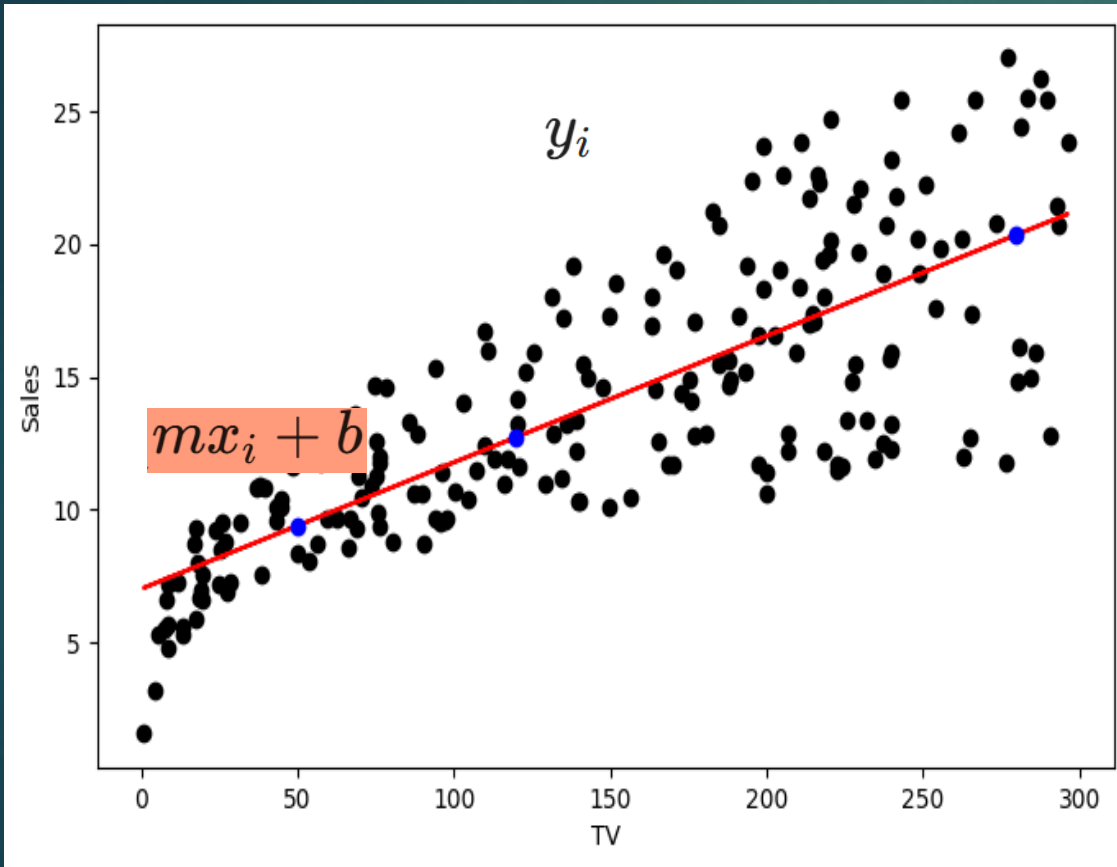
```
optimal_w = sp.solve(diff_L, w)[0]
```

Contents

1. What is Modeling?
2. Python sympy
3. Time & Space (w1)
4. Optimization (w2)
5. Linear Regression (w3) (w4)
6. Optimization of loss function (L1, L2)
7. Short cut algorithm
8. Linear Programming
9. Transportation Problem
10. Montecarlo simulation
11. Random walk simulation
12. Bootstrap
13. Central limit theorem
14. Prince Frog
15. Month Hall
16. Genetic Algorithm
17. Differential Equation

Optimization of Loss Function

40



1. 손실 함수 (Loss Function)

$$E(m, b) = \frac{1}{2n} \sum_{i=1}^n (mx_i + b - y_i)^2$$

(1) m 에 대한 편미분 결과

$$\frac{\partial E}{\partial m} = \frac{1}{n} \sum_{i=1}^n (mx_i + b - y_i)x_i$$

(2) b 에 대한 편미분 결과

$$\frac{\partial E}{\partial b} = \frac{1}{n} \sum_{i=1}^n (mx_i + b - y_i)$$

Contents

1. What is Modeling?
2. Python sympy
3. Time & Space (w1)
4. Optimization (w2)
5. Linear Regression (w3) (w4)
6. Optimization of loss function (L1, L2)
7. Short cut algorithm
8. Linear Programming
9. Transportation Problem
10. Montecarlo simulation
11. Random walk simulation
12. Bootstrap
13. Central limit theorem
14. Prince Frog
15. Month Hall
16. Genetic Algorithm
17. Differential Equation

Optimization of Loss Function

42

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline

rng = np.random.RandomState(1234)
X = 10 * rng.rand(100); y = np.cos(X) + 0.25 * rng.randn(100)
Xfit = np.linspace(0, 10, 100)

models = {
    'Linear':      make_pipeline(LinearRegression()),
    'Polynomial':  make_pipeline(PolynomialFeatures(20), LinearRegression()),
    'Ridge':       make_pipeline(PolynomialFeatures(20), Ridge(alpha=1)),
    'Lasso':       make_pipeline(PolynomialFeatures(20), Lasso(alpha=1)),
}

plt.figure(figsize=(20, 6)); plt.scatter(X, y)

styles = ['r-', 'g-', 'b-', 'k-']

for (name, model), style in zip(models.items(), styles):
    model.fit(X[:, np.newaxis], y)
    plt.plot(Xfit, model.predict(Xfit[:, np.newaxis]), style, label=name)

plt.legend(fontsize=16); plt.ylim(-4, 4)
plt.show()
```

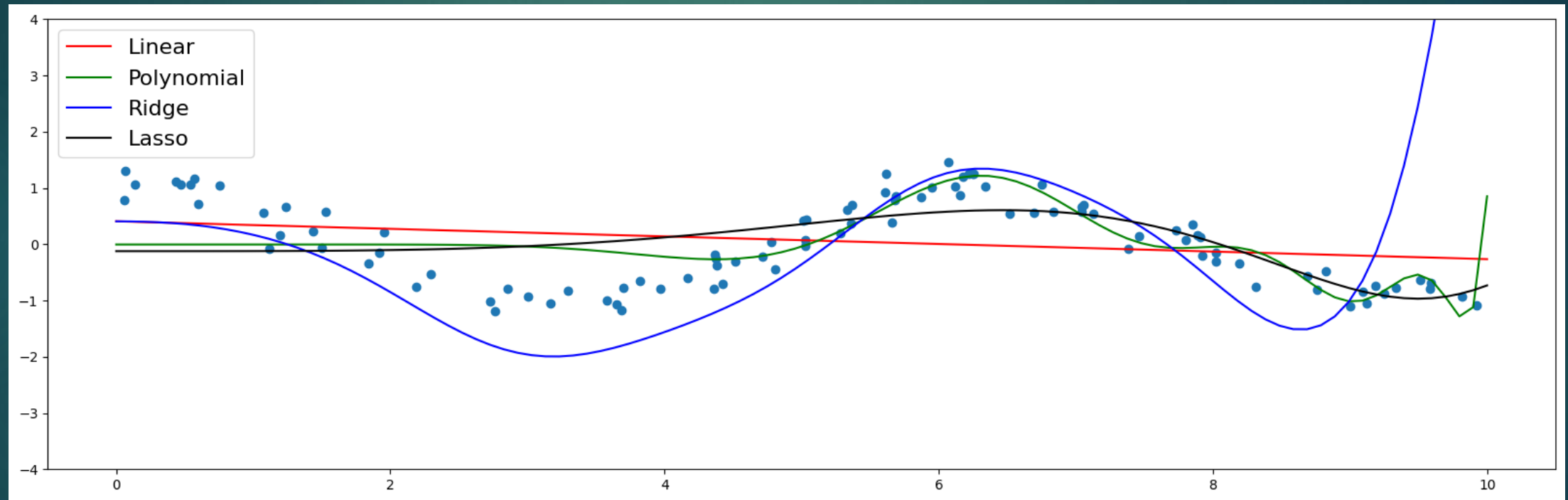


MSE Loss 함수에
L1과 L2 penalty를
부여한 새로운
Loss를 설정하여
pytorch로 코드를
생성하라

Optimization of Loss Function

43

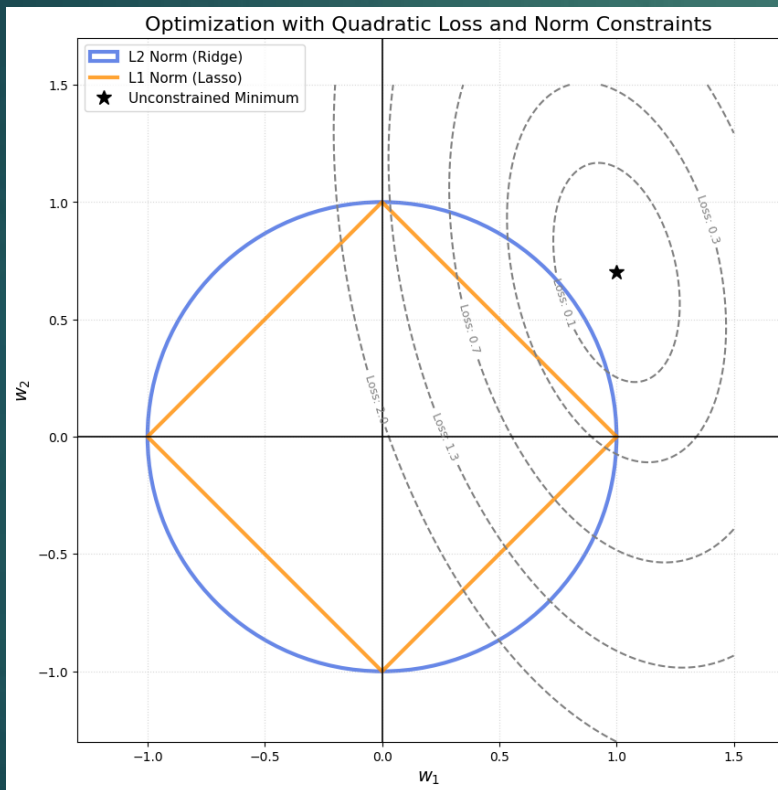
모델이 너무 단순하면(Linear) 못 맞추고, 너무 복잡하면(Polynomial) 예민해지는데, 적절한 규제(Lasso, Ridge)를 주면 훨씬 안정적인 예측 모델이 된다"는 것을 시각적으로 표현



Optimization of Loss Function

44

Lasso(L1 penalty)는 "중요하지 않은 변수를 0으로 만들어 없애 버리는 칼"이고(불필요한 weight를 제거), Ridge(L2 penalty)는 "모든 변수를 안고 가되 그 힘을 골고루 빼놓는 모래주머니(모든 weight를 줄임)"



$$J(w) = \text{MSE} + \alpha \sum_{j=1}^n |w_j|$$

가중치의 절대값 합을 페널티로 사용하여 중요하지 않은 변수의 계수를 정확히 0으로 만들어 변수 선택(Feature Selection) 효과

$$J(w) = \text{MSE} + \alpha \sum_{j=1}^n w_j^2$$

가중치의 제곱 합을 페널티로 사용하여 계수를 완전히 0으로 만들지는 않지만, 전체적으로 크기를 줄여서 모델의 분산(Variance)을 낮추고 과적합을 방지

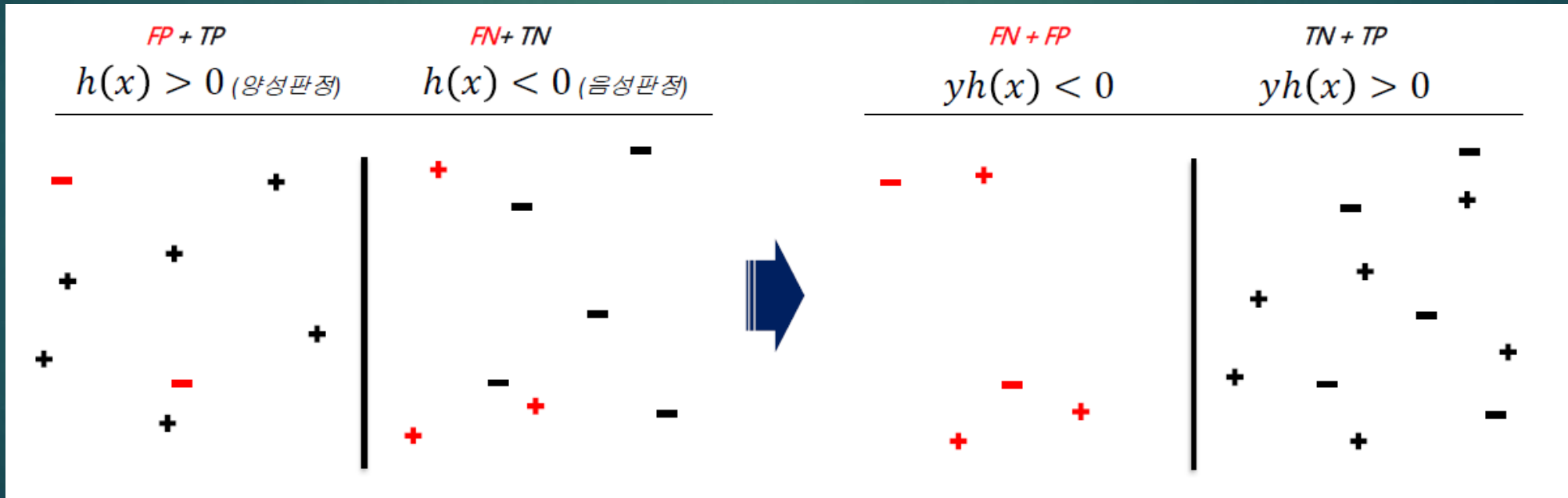
Loss Functions

45

실제 데이터를 정확하게 예측 판단하지 못하는(Loss) 비율

허위음성율(FNR, False Negative Ratio)과 허위양성율(FPR, False Positive Ratio)을 최소화하는 것이 목적

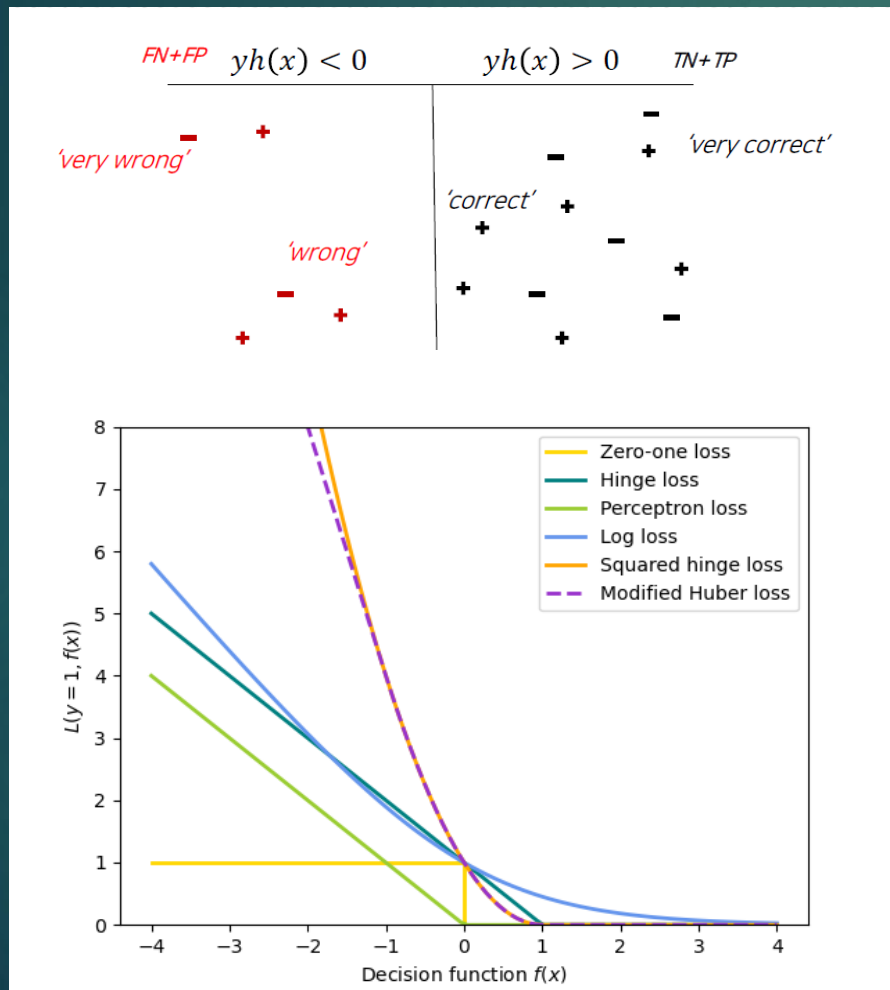
$$\text{Error rate} = \text{FN} + \text{FP} = \frac{1}{n} \sum_{i=1}^n [y_i \neq \text{sign}(h(x))]$$



Loss Functions

46

plot_sgd_loss_functions



Loss	수식	언제 쓰나
MSE	$(y - \hat{y})^2$	기본값, 이상치 없을 때
MAE	$\ y - \hat{y}\ $	이상치 많을 때
Huber	MSE↔MAE 혼합	둘 다 챙기고 싶을 때
Log-Cosh	$\log(\cosh(\Delta))$	Huber보다 부드럽게
Quantile	비대칭 패널티	과소/과대 예측 비용이 다를 때

Contents

1. What is Modeling?
2. Python sympy
3. Time & Space (w1)
4. Optimization (w2)
5. Linear Regression (w3) (w4)
6. Optimization of loss function (L1, L2)
7. Short cut algorithm
8. Linear Programming
9. Transportation Problem
10. Montecarlo simulation
11. Random walk simulation
12. Bootstrap
13. Central limit theorem
14. Prince Frog
15. Month Hall
16. Genetic Algorithm
17. Differential Equation

Short-Cut algorithm

48

서울(Node a)에서 부산(Node h)까지 가는 5가지 경로(Edge)가 있다고 가정하고 경로별로 도착하는 시간은 거리에 비례한다고 가정한다. 어느 경로로 가는 것이 최단거리인지 해를 구하라

Simplest Path DAG from a to h

Nodes and Directed Edges Representing Route Segments

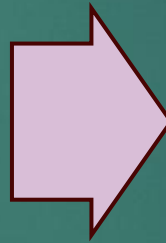
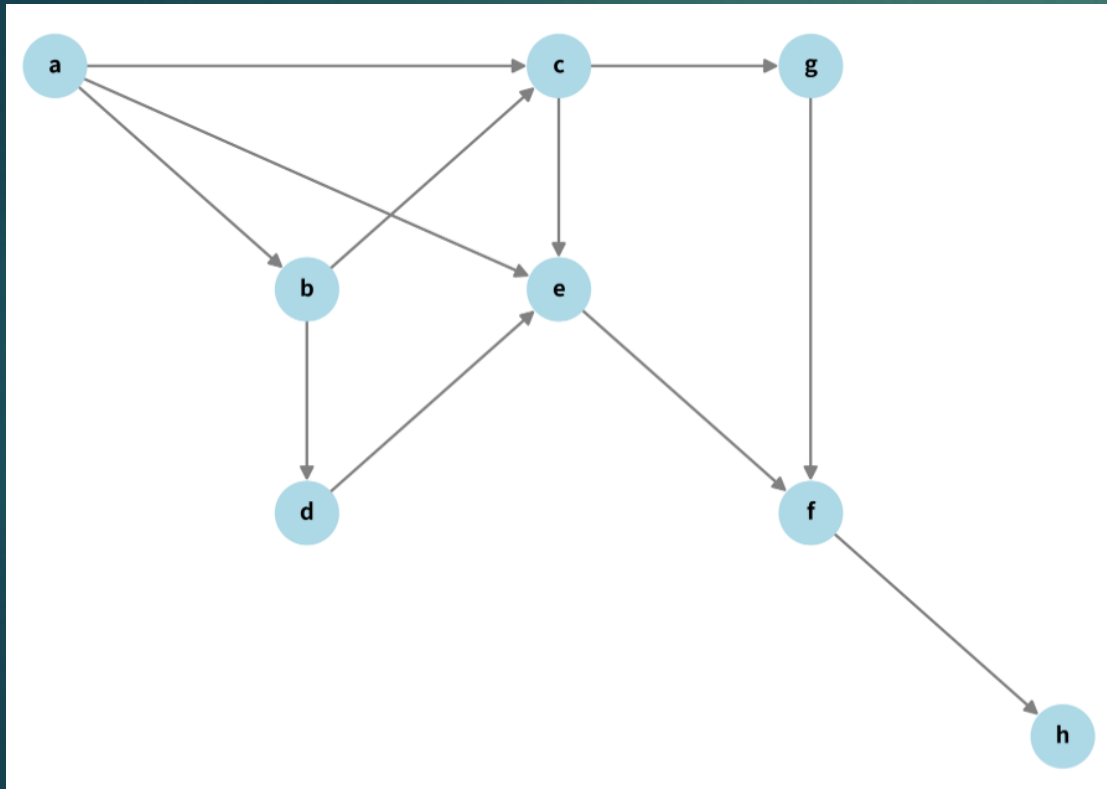


노드	도시	위도 (Latitude)	경도 (Longitude)
a	서울	37.5665	126.9780
b	수원	37.2636	127.0286
c	원주	37.3422	127.9202
d	천안	36.8151	127.1139
e	대전	36.3504	127.3845
f	대구	35.8714	128.6014
g	울산	35.5384	129.3114
h	부산	35.1796	129.0756

Short-Cut algorithm

49

서울(Node a)에서 부산(Node h)까지 가는 5가지 경로(Edge)



8 x 8 인접 행렬 (Adjacency Matrix)

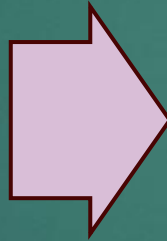
From \ To	a	b	c	d	e	f	g	h
a	0	1	1	0	1	0	0	0
b	0	0	1	1	0	0	0	0
c	0	0	0	0	1	0	1	0
d	0	0	0	0	1	0	0	0
e	0	0	0	0	0	1	0	0
f	0	0	0	0	0	0	0	1
g	0	0	0	0	0	1	0	0
h	0	0	0	0	0	0	0	0

Short-Cut algorithm

50

각 도시의 위경도로 거리 구하기

노드	도시	위도 (Latitude)	경도 (Longitude)
a	서울	37.5665	126.9780
b	수원	37.2636	127.0286
c	원주	37.3422	127.9202
d	천안	36.8151	127.1139
e	대전	36.3504	127.3845
f	대구	35.8714	128.6014
g	울산	35.5384	129.3114
h	부산	35.1796	129.0756



L2 거리 (도시간, 8 x 8 matrix)

② L2 거리 행렬 $D \in \mathbb{R}^{8 \times 8}$

가까움(0) 중간 멀리(3.18)

	a 서울	b 수원	c 원주	d 천안	e 대전	f 대구	g 울산	h 부산
a 서울	0.0000	0.3071	0.9685	0.7636	1.2822	2.3471	3.0916	3.1776
b 수원	0.3071	0.0000	0.8951	0.4565	0.9801	2.1005	2.8614	2.9212
c 원주	0.9685	0.8951	0.0000	0.9633	1.1272	1.6209	2.2780	2.4519
d 천안	0.7636	0.4565	0.9633	0.0000	0.5377	1.7616	2.5415	2.5540
e 대전	1.2822	0.9801	1.1272	0.5377	0.0000	1.3078	2.0910	2.0568
f 대구	2.3471	2.1005	1.6209	1.7616	1.3078	0.0000	0.7842	0.8387
g 울산	3.0916	2.8614	2.2780	2.5415	2.0910	0.7842	0.0000	0.4293
h 부산	3.1776	2.9212	2.4519	2.5540	2.0568	0.8387	0.4293	0.0000

Short-Cut algorithm

51

인접행렬과 도시간 거리 행렬을 내적하면 각 행은 출발도시에서 각 열 도시까지의 도달하는데 가능한 모든 경로에 해당되는 누적 거리, 노란색 박스는 서울(Node a)에서 각 도시까지 가는데 걸리는 누적합산 거리

Adjacency Matrix
(8, 8)

@

도시간 L2 거리
(8, 8)

= 이웃 거리의 합산 → 최단 경로와 무관

```
[[0., 1., 1., 0., 1., 0., 0., 0.],
 [0., 0., 1., 1., 0., 0., 0., 0.],
 [0., 0., 0., 0., 1., 0., 1., 0.],
 [0., 0., 0., 0., 1., 0., 0., 0.],
 [0., 0., 0., 0., 0., 1., 0., 0.],
 [0., 0., 0., 0., 0., 0., 0., 1.],
 [0., 0., 0., 0., 0., 1., 0., 0.],
 [0., 0., 0., 0., 0., 0., 0., 0.]
```

@

```
[[0., 0.31, 0.97, 0.76, 1.28, 2.35, 3.09, 3.18],
 [0.31, 0., 0.9, 0.46, 0.98, 2.1, 2.86, 2.92],
 [0.97, 0.9, 0., 0.96, 1.13, 1.62, 2.28, 2.45],
 [0.76, 0.46, 0.96, 0., 0.54, 1.76, 2.54, 2.55],
 [1.28, 0.98, 1.13, 0.54, 0., 1.31, 2.09, 2.06],
 [2.35, 2.1, 1.62, 1.76, 1.31, 0., 0.78, 0.84],
 [3.09, 2.86, 2.28, 2.54, 2.09, 0.78, 0., 0.43],
 [3.18, 2.92, 2.45, 2.55, 2.06, 0.84, 0.43, 0.]]
```

=

```
[[2.56 1.88 2.02 1.96 2.11 5.03 7.23 7.43]
 [1.73 1.35 0.96 0.96 1.66 3.38 4.82 5.01]
 [4.37 3.84 3.41 3.08 2.09 2.09 2.09 2.49]
 [1.28 0.98 1.13 0.54 0. 1.31 2.09 2.06]
 [2.35 2.1 1.62 1.76 1.31 0. 0.78 0.84]
 [3.18 2.92 2.45 2.55 2.06 0.84 0.43 0. ]
 [2.35 2.1 1.62 1.76 1.31 0. 0.78 0.84]
 [0. 0. 0. 0. 0. 0. 0. 0. ]]
```

Contents

1. What is Modeling?
2. Python sympy
3. Time & Space (w1)
4. Optimization (w2)
5. Linear Regression (w3) (w4)
6. Optimization of loss function (L1, L2)
7. Short cut algorithm
8. Linear Programming
9. Transportation Problem
10. Montecarlo simulation
11. Random walk simulation
12. Bootstrap
13. Central limit theorem
14. Prince Frog
15. Month Hall
16. Genetic Algorithm
17. Differential Equation

아래 문제 → 선형계획법으로 이익을 최대화하는 Optimization

- 목표:
 - 의자(x_1)와 책상(x_2)을 생산하는 공장에서 이익을 최대화하는 의자와 책상의 생산량은?
- 이익:
 - 의자 한 개당 3만 원, 책상 한 개당 5만 원의 이익
- 자원 제약:
 - 목재: 의자는 2단위, 책상은 3단위가 필요하며 총 30단위 까지만 사용 가능
 - 노동 시간: 의자는 1시간, 책상은 2시간이 필요하며 총 16시간 까지만 가능

Linear Programming

54

- 목적 함수 (Maximize):

$$z = 3x_1 + 5x_2$$

- 제약 조건 (Subject to):

1. $2x_1 + 3x_2 \leq 30$ (목재)

2. $x_1 + 2x_2 \leq 16$ (노동)

3. $x_1, x_2 \geq 0$ (비음 조건)

$$\max \quad z = \begin{bmatrix} 3 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$\text{s.t.} \quad \begin{bmatrix} 2 & 3 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \leq \begin{bmatrix} 30 \\ 16 \end{bmatrix}$$

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \geq \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Linear Programming

55

```
import numpy as np
from scipy.optimize import linprog

# 1. 목적 함수 계수 (Maximize 3x1 + 5x2 => Minimize -3x1 - 5x2)
c = [-3, -5]

# 2. 제약 조건 행렬 A와 벡터 b (Ax <= b)
# [2x1 + 3x2 <= 30]
# [1x1 + 2x2 <= 16]
A = [[2, 3], [1, 2]]
b = [30, 16]

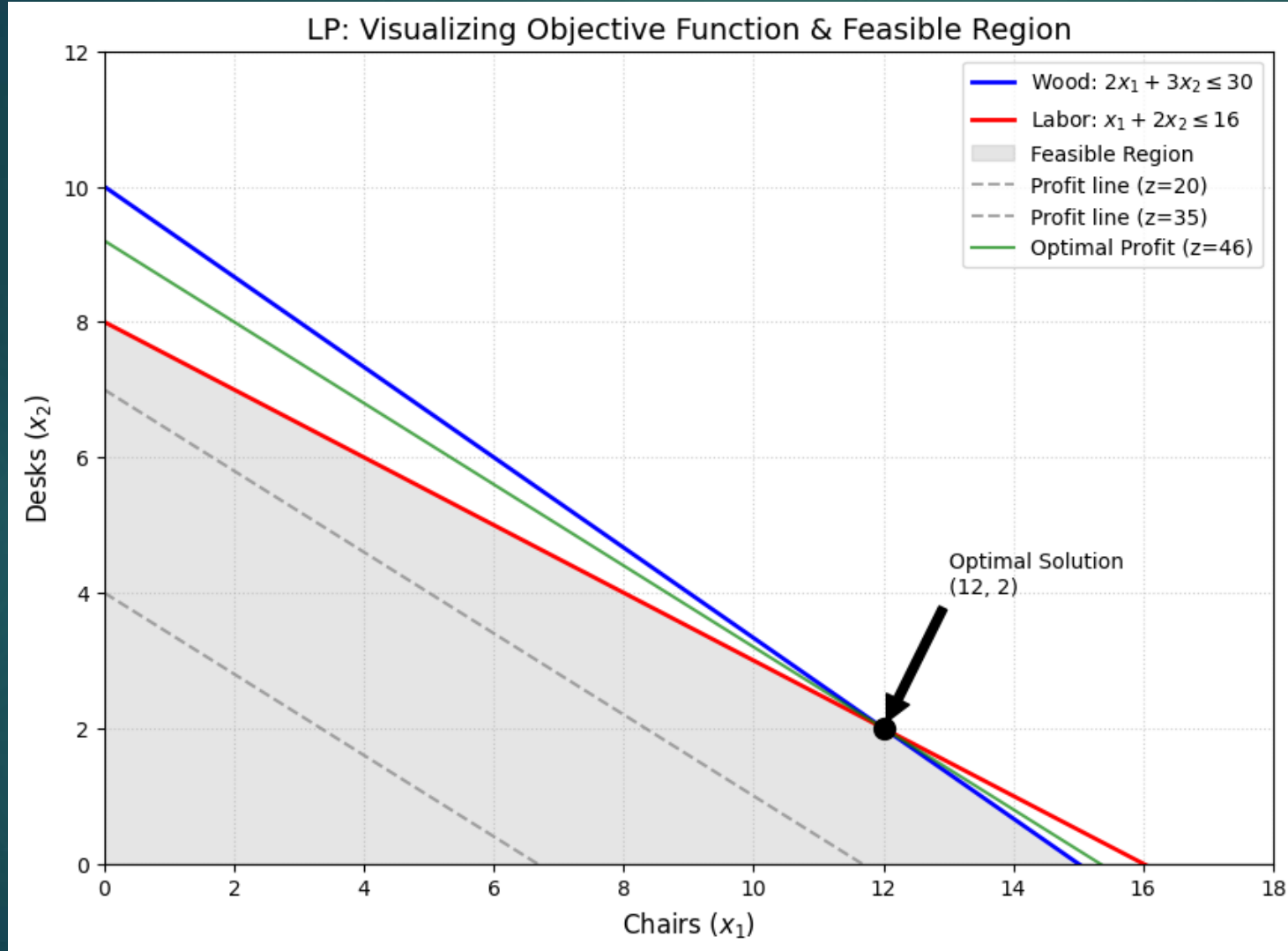
# 3. 변수 범위 (x1 >= 0, x2 >= 0)
x_bounds = (0, None)
y_bounds = (0, None)

# 4. 선형계획법 실행
res = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, y_bounds], method='highs')

print(f"최적 생산량: 의자 {res.x[0]:.1f}개, 책상 {res.x[1]:.1f}개")
print(f"최대 이익: {-res.fun:.1f}만원")
```

Linear Programming

56



Contents

1. What is Modeling?
2. Python sympy
3. Time & Space (w1)
4. Optimization (w2)
5. Linear Regression (w3) (w4)
6. Optimization of loss function (L1, L2)
7. Short cut algorithm
8. Linear Programming
9. Transportation Problem
10. Montecarlo simulation
11. Random walk simulation
12. Bootstrap
13. Central limit theorem
14. Prince Frog
15. Month Hall
16. Genetic Algorithm
17. Differential Equation

Transportation Problems

58

수송 문제 데이터 테이블 (비용 및 수급 현황)

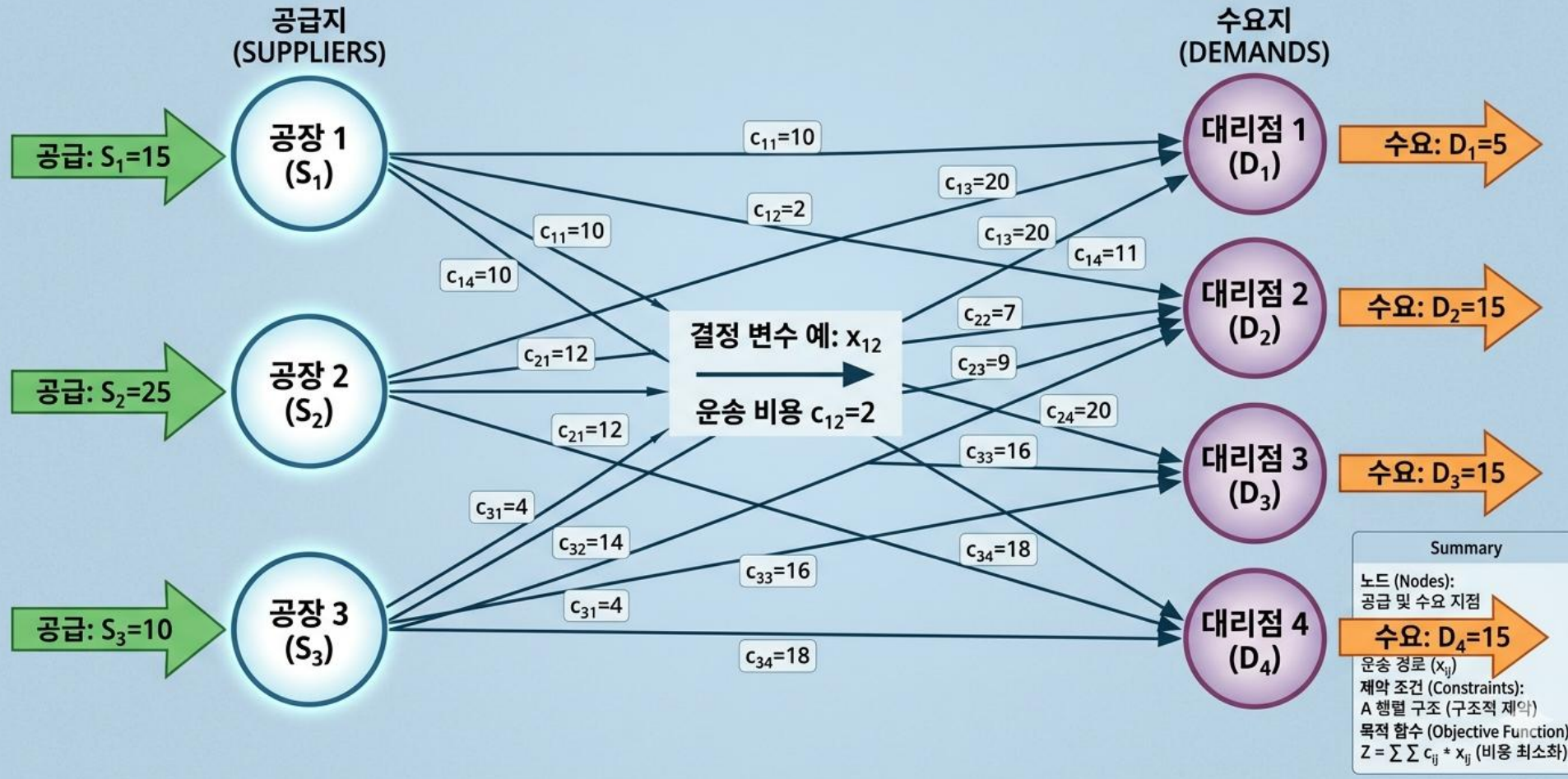
각 셀의 숫자는 단위당 운송 비용을 의미하며, 마지막 열과 마지막 행은 각각의 공급 능력과 수요량

출발(공급) \ 도착(수요)	대리점 1 (D1)	대리점 2 (D2)	대리점 3 (D3)	대리점 4 (D4)	공급 가능량 (Si)
공장 1 (S1)	10	2	20	11	15
공장 2 (S2)	12	7	9	20	25
공장 3 (S3)	4	14	16	18	10
요구 수요량 (Dj)	5	15	15	15	총합: 50

Transportation Problems

59

수송 문제 데이터 테이블의 노드 및 엣지 시각화



Transportation Problems

60

$$\text{Minimize } Z = c^T x$$

(여기서 $c = [10, 2, 20, 11, 12, 7, 9, 20, 4, 14, 16, 18]^T$)

전체 행렬식: $Ax = b$

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_{11} \\ x_{12} \\ x_{13} \\ x_{14} \\ x_{21} \\ x_{22} \\ x_{23} \\ x_{24} \\ x_{31} \\ x_{32} \\ x_{33} \\ x_{34} \end{bmatrix} = \begin{bmatrix} 15 \\ 25 \\ 10 \\ 5 \\ 15 \\ 15 \\ 15 \\ 15 \end{bmatrix}$$

선형계획법(Linear Programming)

최적화 상태: Optimal

Route_공장1_대리점2 = 5.0

Route_공장1_대리점4 = 10.0

Route_공장2_대리점2 = 10.0

Route_공장2_대리점3 = 15.0

Route_공장3_대리점1 = 5.0

Route_공장3_대리점4 = 5.0

최소 총 운송 비용 (Z) = 435.0

Contents

1. What is Modeling?
2. Python sympy
3. Time & Space (w1)
4. Optimization (w2)
5. Linear Regression (w3) (w4)
6. Optimization of loss function (L1, L2)
7. Short cut algorithm
8. Linear Programming
9. Transportation Problem
10. Montecarlo simulation
11. Random walk simulation
12. Bootstrap
13. Central limit theorem
14. Prince Frog
15. Month Hall
16. Genetic Algorithm
17. Differential Equation

Montecarlo Simulation

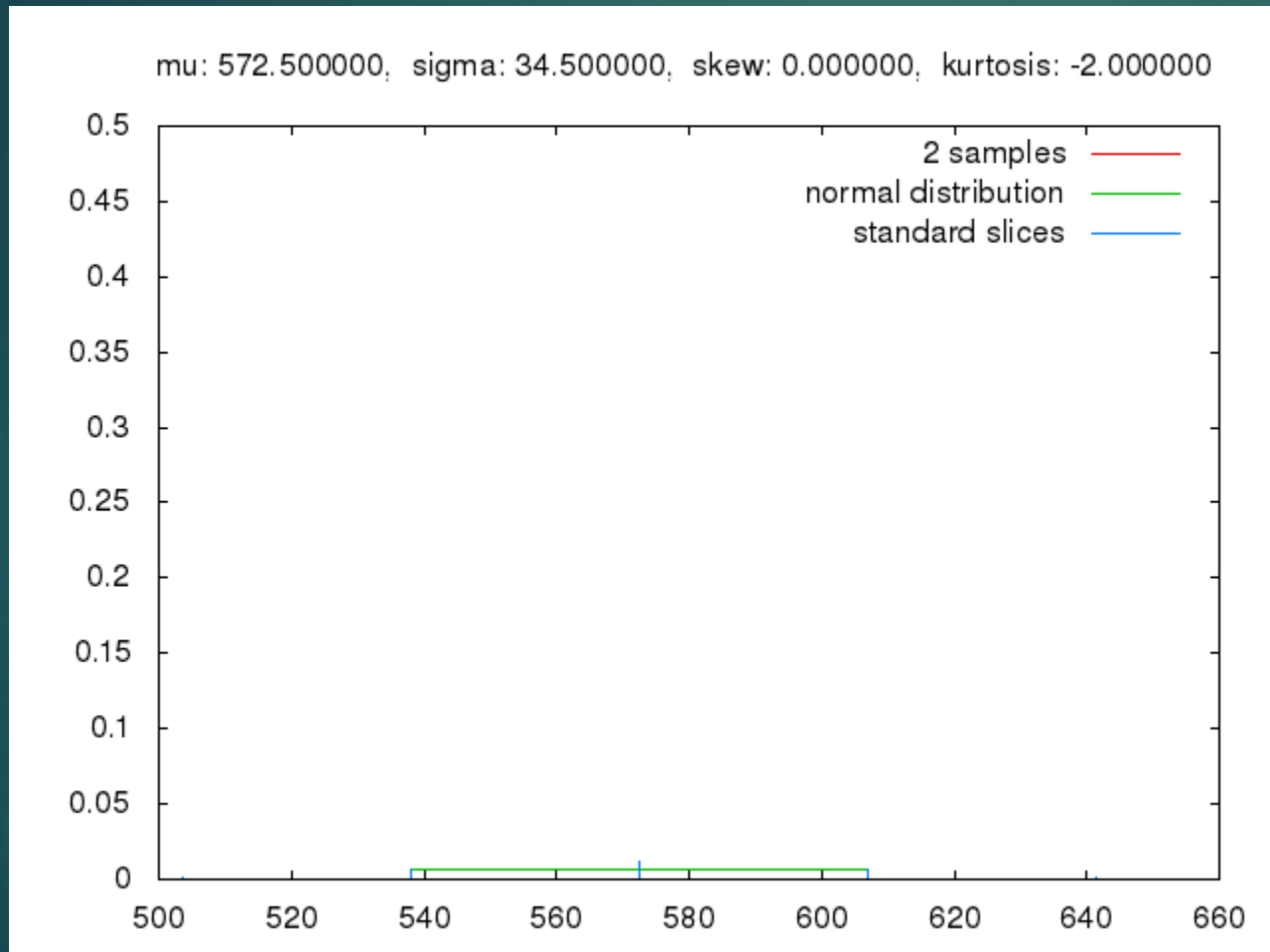
62

- ❖ 수많은 무작위(Random) 숫자를 대입해 보면서, 발생할 수 있는 복잡한 결과의 확률적 분포를 알아내는 기법
- ❖ 수학 공식이나 방정식으로 명확한 답을 내리기 어려울 정도로 불확실성이 크거나 변수가 많을 때, 컴퓨터에게 "주사위를 수만 번 던지게 해서" 그 통계적인 결과로 정답을 찾아가는 기법
- ❖ 아주 똑똑한 노가다 알고리즘(Brute Force Algorithm)
- ❖ **Monte Carlo methods**, also called the **Monte Carlo experiments** or **Monte Carlo simulations**, are a broad class of computational algorithms based on repeated random sampling for obtaining numerical results. The underlying concept is to use randomness to solve deterministic problems. → (Wikipedia)

Montecarlo Simulation

63

[scripts/montecarlo_method.ipynb](#)



[computational algorithms](#)

based on **repeated random sampling**
for obtaining numerical results.

Simulation

64

균등분포(Uniform Distribution) 확률 분석 : 통학 버스가 정확히 15분 간격으로 도착할 경우 임의의 시간에 정류장에 도착한다고 가정하면 평균 기다리는 시간(W)과 10분 이상 기다리는 확률을 구하라

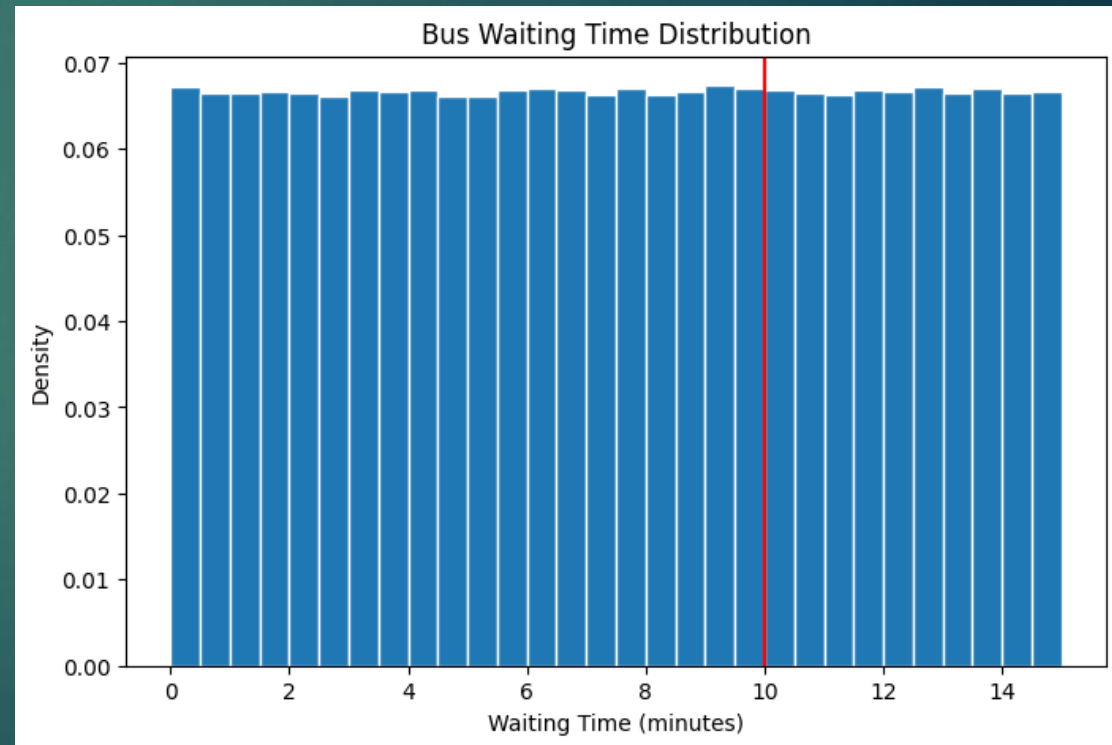
$$W \sim \text{Uniform}(0, 15)$$

```
import numpy as np
import matplotlib.pyplot as plt

sims = 1000000
wait = np.random.uniform(0, 15, sims)
mean_wait = np.mean(wait)
prob_10 = np.mean(wait >= 10)

plt.figure(figsize=(8,5))
plt.hist(wait, bins=30, density=True, edgecolor='white')
plt.axvline(10, color='red')
plt.title("Bus Waiting Time Distribution")
plt.xlabel("Waiting Time (minutes)")
plt.ylabel("Density")

plt.show()
```



Simulation

65

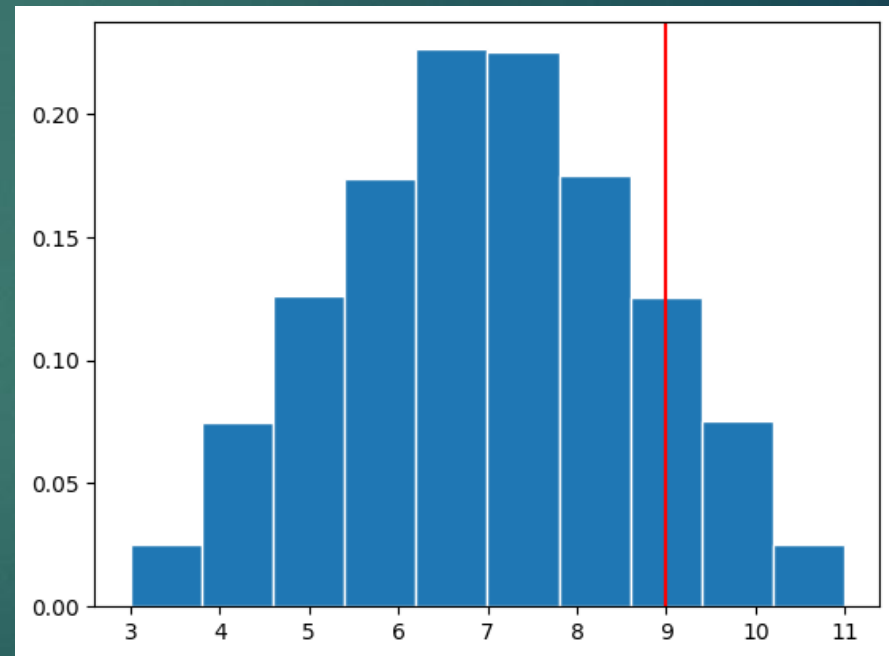
균등분포(Uniform Distribution) 합의 확률 분석 : 어떤 작업은 두 단계 A 와 B 로 이루어지는데 단계 A 의 소요시간은 1~5 사이에서, 단계 B 의 소요시간은 2~6 사이에서 각각 균등하게 발생할 경우 전체 작업 시간($A + B$) 이 9시간 이상일 확률을 구하라

```
import numpy as np
sims = 1000000

A = np.random.uniform(1, 5, sims)
B = np.random.uniform(2, 6, sims)

duration = A + B

import matplotlib.pyplot as plt
plt.figure()
plt.hist(duration, density = True, edgecolor='white')
plt.axvline(9, color='r')
plt.show()
print((duration > 9).sum()/sims)
```



Simulation

66

A 와 B 확률변수의 합의 확률변수 $A + B$ 합성곱(convolution)은 $\rightarrow$ CNN(Convolutional Neural Network)

	1	2	3	4	5
2	3	4	5	6	7
3	4	5	6	7	8
4	5	6	7	8	9
5	6	7	8	9	10
6	7	8	9	11	11

합성곱 행렬 M (9x5):

```
[[0.201 0. 0. 0. 0. ]
 [0.203 0.201 0. 0. 0. ]
 [0.204 0.203 0.201 0. 0. ]
 [0.194 0.204 0.203 0.201 0. ]
 [0.199 0.194 0.204 0.203 0.201]
 [0. 0.199 0.194 0.204 0.203]
 [0. 0. 0.199 0.194 0.204]
 [0. 0. 0. 0.199 0.194]
 [0. 0. 0. 0. 0.199]]
```

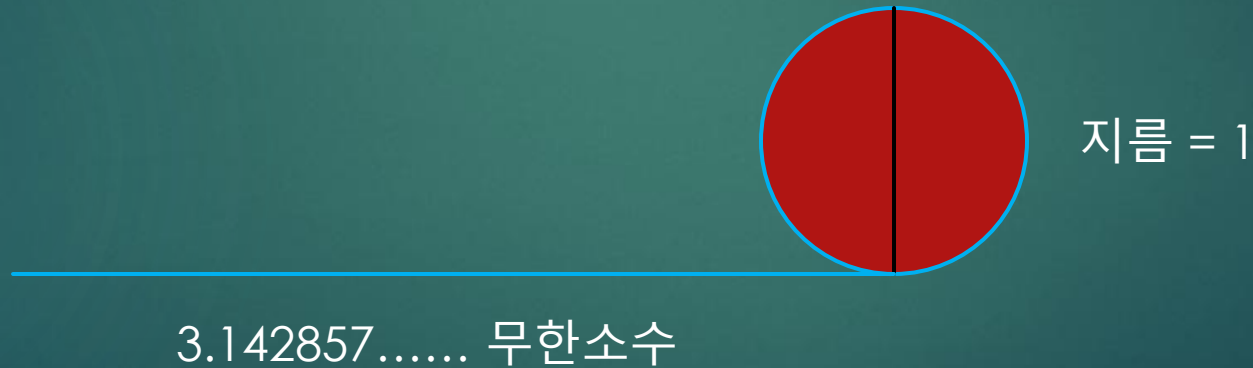
z	행렬곱	시뮬레이션
$z=3$	0.0412	0.0386
$z=4$	0.0819	0.0829
$z=5$	0.1222	0.1236
$z=6$	0.1604	0.1595
$z=7$	0.2000	0.2076
$z=8$	0.1588	0.1529
$z=9$	0.1181	0.1173
$z=10$	0.0778	0.0767
$z=11$	0.0396	0.0409

행렬곱 합계: 1.0000
np.convolve 일치: True

Simulation

67

- ❖ 지금이 1인 원의 둘레가 π 이다. 원의 둘레를 (0.5, 0)을 시작벡터로 부터 아주 작은 단위 (epsilon=0.00001)로 이동(미분)한 새로운 벡터와의 L2 거리를 무수히(100_000) 생성하여 누적거리를 구하는(적분) 모의실험으로 원주율 π 를 구하라
 - ❖ 단 : L2 거리(math.sqrt) 이외 원의 방정식, np.pi 등 어떠한 공식도 사용하지 마라
- ❖ 수치해석(Numerical Analysis) 만을 사용하여 원주율 π 를 구하는 코드를 제시하라



Simulation

68

```
import math

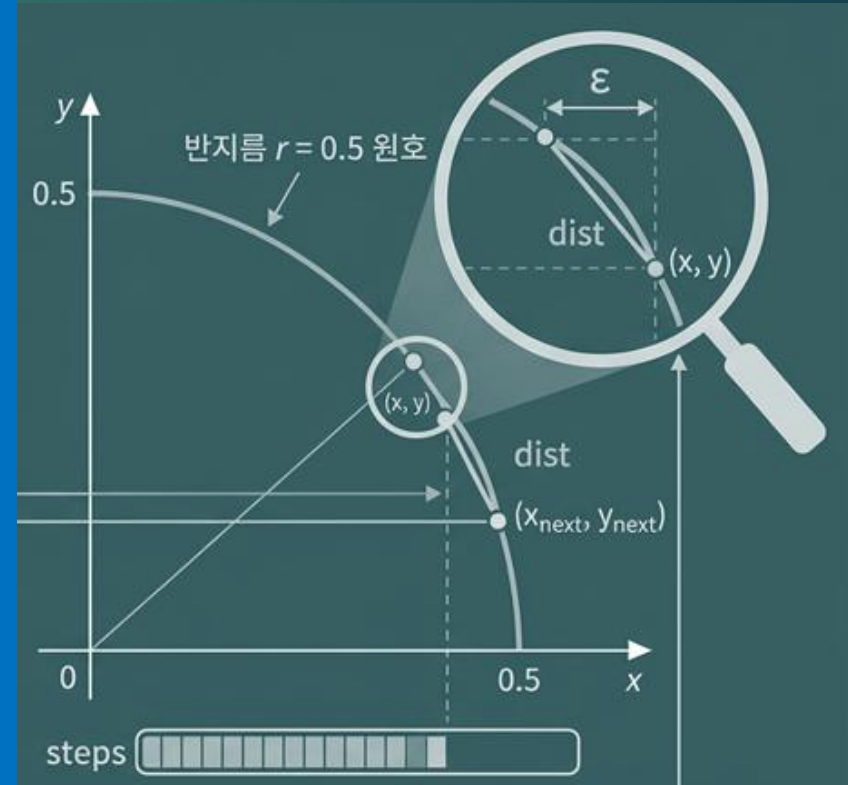
epsilon = 0.000001
total_distance = 0; steps = 0
x = 0.5 # (1, 0)에서 시작

while x > 0.0:
    x_next = max(x - epsilon, 0.0)

    # 단위원 위의 y값:  $y = \sqrt{1 - x^2}$ 
    y1 = math.sqrt(0.5**2 - x**2)
    y2 = math.sqrt(0.5**2 - x_next**2)

    # 두 점 사이의 거리
    dist = math.sqrt((x_next - x)**2 + (y2 - y1)**2)
    total_distance += dist
    steps += 1; x = x_next

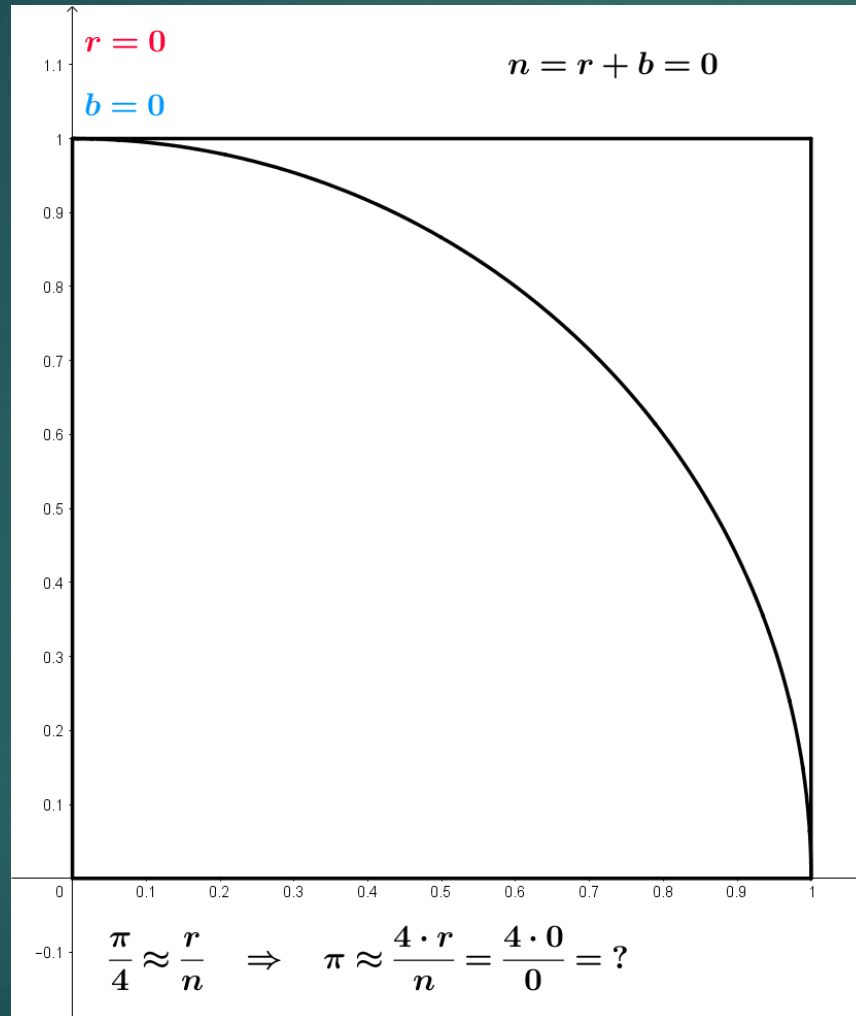
print(f"총 단계 수: {steps:,}")
print(f"누적 거리 합계: {total_distance:.10f}")
print(f"원주율 : {total_distance*4:.10f}")
```



```
epsilon : 0.000001
steps : 0
total_distance : 0.000000967
π : 1.28773128424 입력
```

Simulation

69



Simulation

70

- ❖ L2 거리(피타고라스 정리)로 원주율을 구하는 모의실험 코드
 - ❖ 원의 넓이는 πr^2 ($\pi = \text{np.pi}$)

```
Import numpy as np

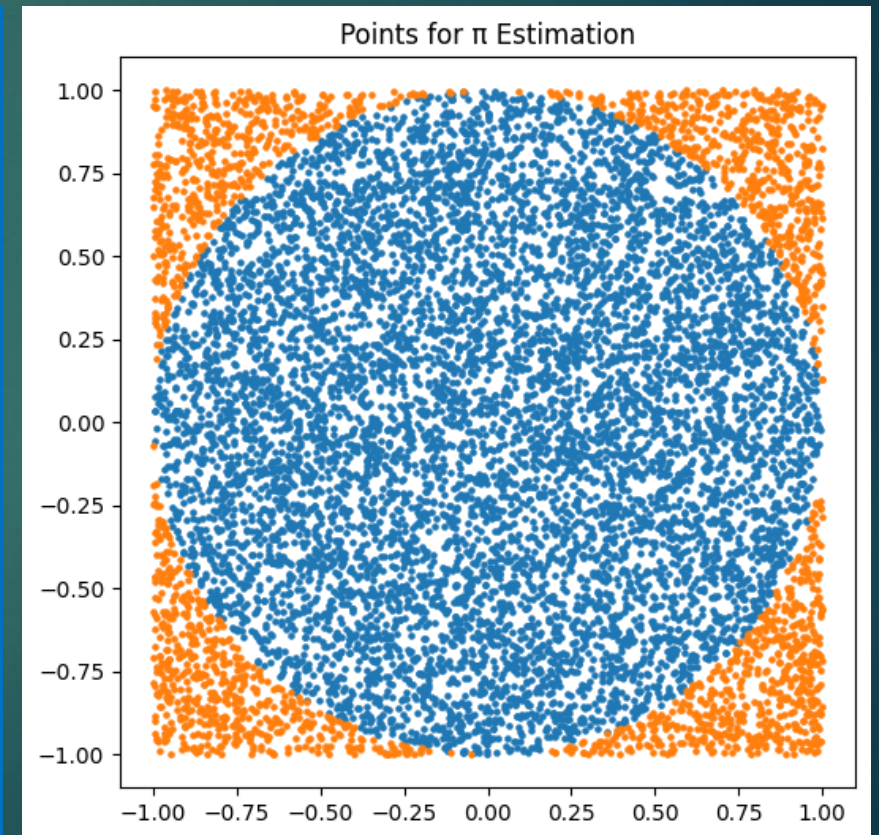
N = 10_000_000 # 점의 개수 (많을수록 정확)

# 1. -1~1 사이의 무작위 x, y 좌표 생성
X = np.random.uniform(-1, 1, n); y = np.random.uniform(-1, 1, n)

# 2. 원점으로부터의 거리가 1 이하인 점 = 원 안의 점
#  $x^2 + y^2 \leq 1$  (피타고라스,  $\pi$  사용 안 함!)
inside = np.sum(x**2 + y**2 <= 1)

# 3.  $\pi$  추정
pi_estimate = 4 * inside / N

print(f"점의 개수      : {N:,}개")
print(f"원 안의 점      : {inside:,}개")
print(f"추정된  $\pi$          : {pi_estimate:.6f}")
print(f"오차              : {abs(pi_estimate - np.pi):.6f}")
```



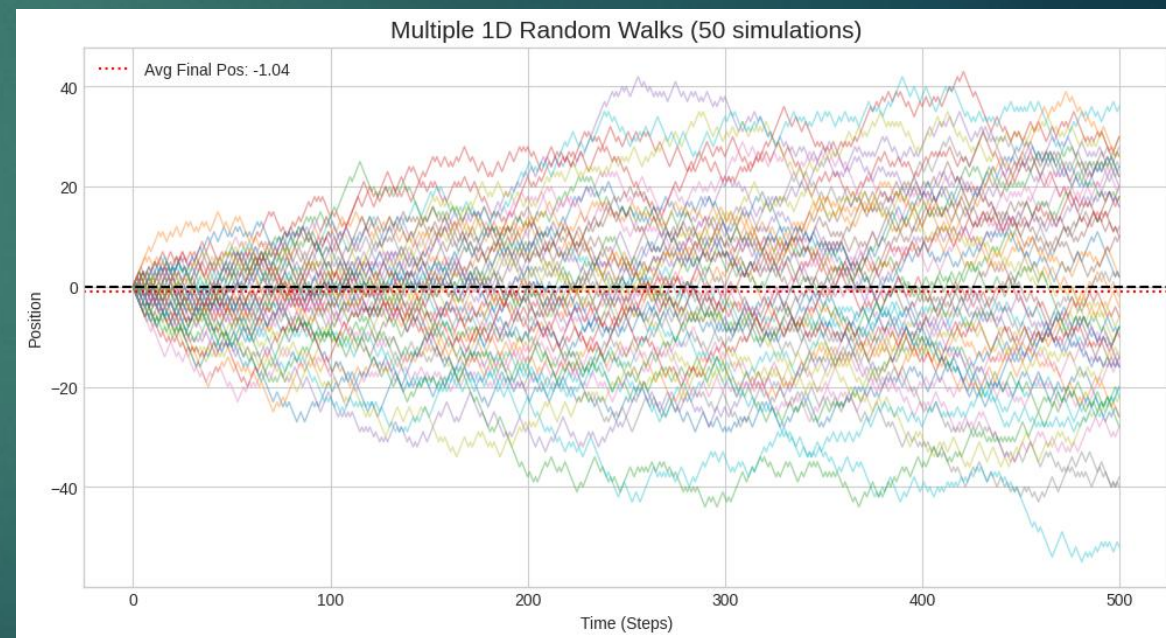
Contents

1. What is Modeling?
2. Python sympy
3. Time & Space (w1)
4. Optimization (w2)
5. Linear Regression (w3) (w4)
6. Optimization of loss function (L1, L2)
7. Short cut algorithm
8. Linear Programming
9. Transportation Problem
10. Montecarlo simulation
11. Random walk simulation
12. Bootstrap
13. Central limit theorem
14. Prince Frog
15. Month Hall
16. Genetic Algorithm
17. Differential Equation

Random walk simulation

72

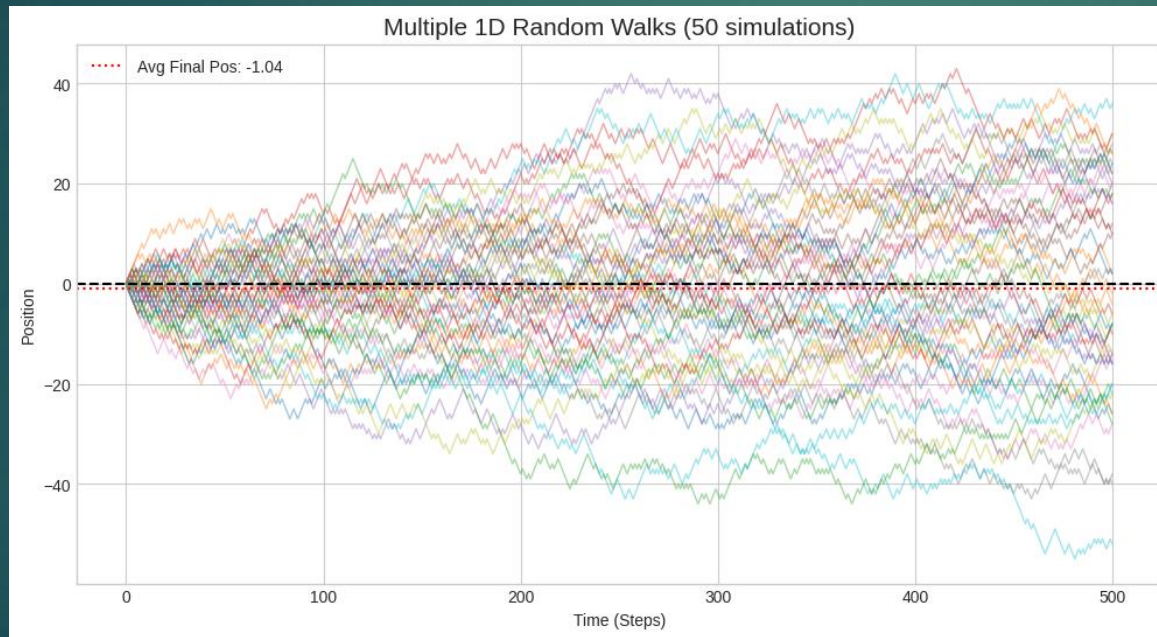
- ❖ 1차원 단순 랜덤 워크 (1D Simple Random Walk) – [snippet](#)
- ❖ 동전 던지기처럼 앞(+1) 또는 뒤(-1)로만 움직이는 가장 기본적인 형태
 - ❖ 이산적 이동으로 매 스텝마다 이동 거리가 항상 ± 1
 - ❖ 움직임이 고정된 크기로 제한됨
 - ❖ 격자 위를 한 칸씩 움직이는 형태



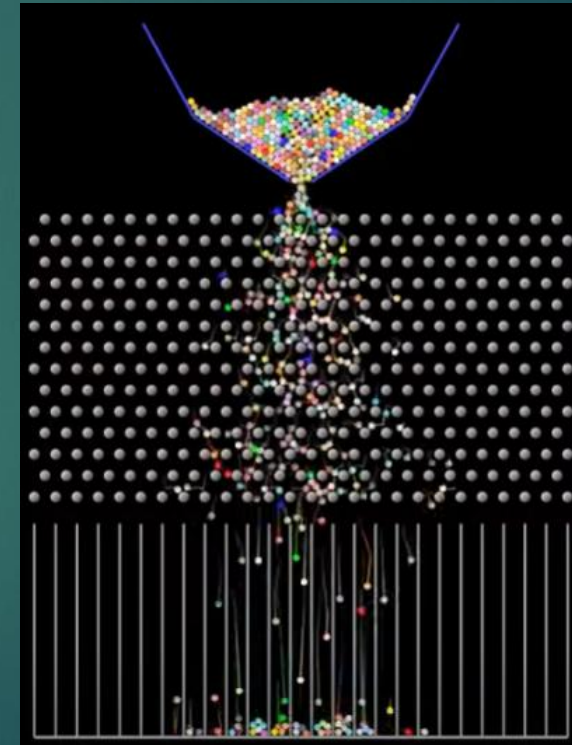
Random walk simulation

73

- ❖ 1차원 아래 모의실험을 베르누이, 이항분포, Galton Board와 비교하여 이해하자
 - ❖ 구슬 ?, single random walk ?, 여러 명이 술이 취해 걸으면 최종 어디에 많이 모일까?



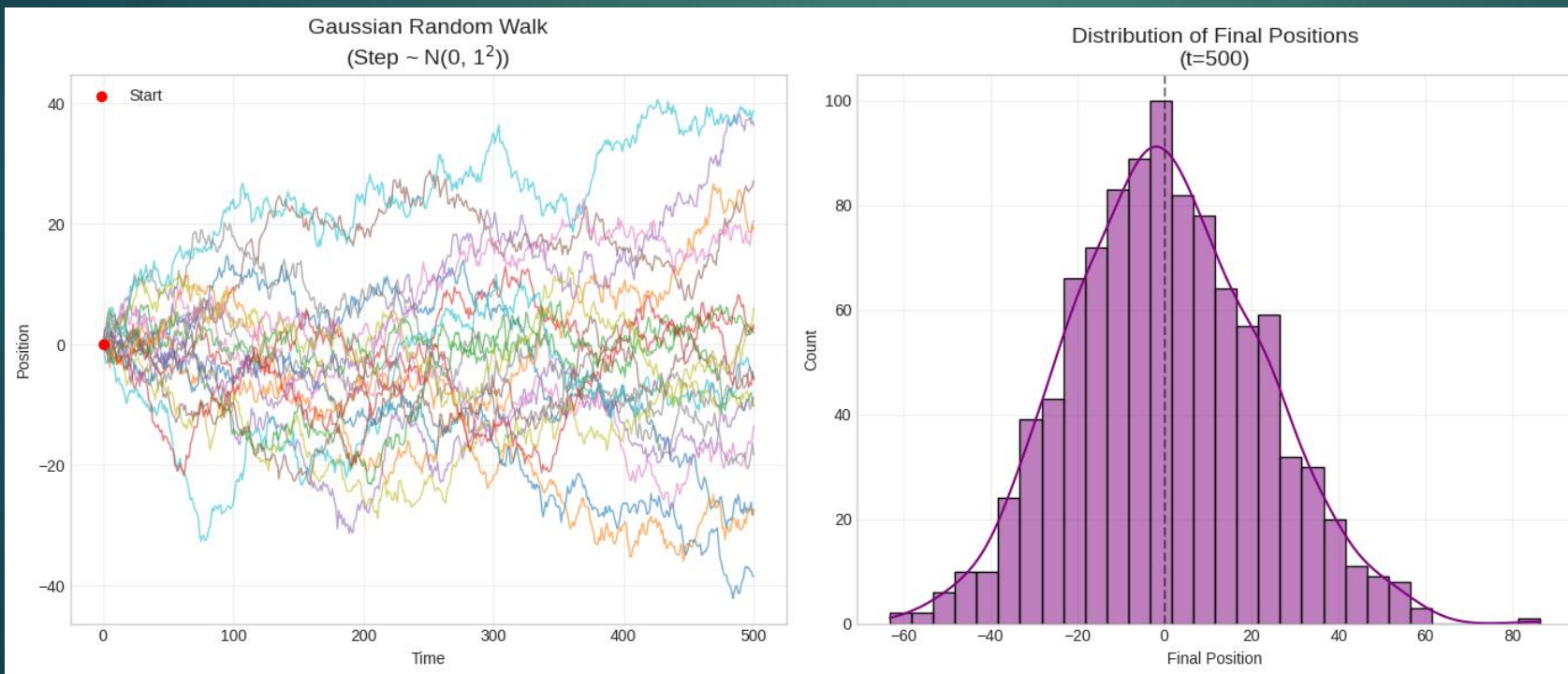
=



Random walk simulation

74

- 2차원 Gaussian 랜덤 워크 : 정규분포(Normal Distribution)를 이용한 랜덤 워크는 가우시안 랜덤 워크(Gaussian Random Walk)라고 하며, 수학적으로는 브라운 운동(Brownian Motion)의 기초가 되는 모델
 - 연속적 이동(정규분포 기반), 이동 거리가 정규분포(종 모양)를 따름
 - 대부분은 작게 움직이고, 가끔 크게 이동하는 경우도 발생하며 실제 세계 물리현상과 유사



- ✓ 미세 입자의 확산(브라운 운동)
- ✓ 주식 가격 변동
- ✓ 열 운동, 분자 운동의 시뮬레이션

Contents

1. What is Modeling?
2. Python sympy
3. Time & Space (w1)
4. Optimization (w2)
5. Linear Regression (w3) (w4)
6. Optimization of loss function (L1, L2)
7. Short cut algorithm
8. Linear Programming
9. Transportation Problem
10. Montecarlo simulation
11. Random walk simulation
12. Bootstrap
13. Central limit theorem
14. Prince Frog
15. Month Hall
16. Genetic Algorithm
17. Differential Equation

Bootstrap sampling

76

- ❖ 한 학급에 100명($N=100$)이 있는 반을 상상해 보세요. 선생님이 무작위로 1 명을 뽑고 선물을 주고, 다시 전체 100명에서 1명을 뽑습니다.
 - ❖ 100번 반복해서 선물을 주게 될 경우 내가 선물을 못박을 확률은?
 - ❖ 200번을 반복했는데 내가 못 받을 확률은?
 - ❖ 300번을 반복했는데 내가 못 받을 확률은?
- ❖ Bootstrap Sample이란 무엇인가?
- ❖ N 이 무한대이고 무한으로 반복하면 내가 못 받을 확률은?

Bootstrap sampling

77

- ❖ 한 학급에 100명($N=100$)이 있는 반을 상상해 보세요. 선생님이 무작위로 1 명을 뽑고 선물을 주고, 다시 전체 100명에서 1명을 뽑습니다.
 - ❖ 100번 반복해서 선물을 주게 될 경우 내가 선물을 못박을 확률은?
 - ❖ 200번을 반복했는데 내가 못 받을 확률은?
 - ❖ 300번을 반복했는데 내가 못 받을 확률은?
- ❖ N 이 무한대이고 무한으로 반복하면 내가 못 받을 확률은?

Bootstrap sampling

78

- ▶ 전체 데이터셋의 크기가 N 일 때, 원본 데이터에서 한 번 뽑은 데이터를 다시 넣고(복원) 무작위로 N 번 추출하여 만든 새로운 샘플
 - ▶ 중복 허용: 동일한 데이터가 한 샘플 내에 여러 번 포함될 수 있음
 - ▶ 미포함 데이터: 확률적으로 원본 데이터의 약 $36.8\%(1/e)$ 는 샘플에 포함되지 않는데, 이를 OOB(Out-Of-Bag) 데이터라고 부르며 모델 검증용으로 활용(이 수치는 앞서 질문하신 '선물을 못 받을 확률'과 동일한 원리) → Randomforest 모델의 OOB

- 표본의 수가 적거나 추가 자료를 얻기 어려워 통계가설이 부재
- 재표본 추출(Resampling) = 부트스트래핑(Bootstrapping)
- 표본 N 개에서 k 개를 복원 추출(중복 선택)하여 모수 추정치의 표준 오차를 줄이게 된다.
 - $N=100$ 개인 표본에서 200회 재표본(재표본 수는 100개) 추출하는 부트스트래핑을 적용하면 관측치 수가 100개인 표본 200개를 생성



Contents

1. What is Modeling?
2. Python sympy
3. Time & Space (w1)
4. Optimization (w2)
5. Linear Regression (w3) (w4)
6. Optimization of loss function (L1, L2)
7. Short cut algorithm
8. Linear Programming
9. Transportation Problem
10. Montecarlo simulation
11. Random walk simulation
12. Bootstrap
13. Central limit theorem
14. Prince Frog
15. Month Hall
16. Genetic Algorithm
17. Differential Equation

Central Limit Theorem

80

- ❖ 중심극한정리(데이터는 불규칙하지만, 평균은 예측 가능하다 → 그래서 학습이 가능하다)가 머신러닝에 접목하는 내용을 모의실험으로 파이썬 코드로 제시하라
- ❖ 모델 학습의 안정성 확보
 - ❖ MSE , Cross-Entropy, Log-loss
- ❖ Mini-batch 학습이 가능한 이유
 - ❖ Mini-batch = 작은 샘플 평균, CLT없이도 mini batch 도 있음
- ❖ 교차검증을 통한 모델의 평가
 - ❖ 평균 accuracy/loss, 교차검증 평균 등 CLT 적용으로 이 평균들은 → 정규분포 근사 → 신뢰구간 → 통계적 비교 가능
- ❖ A/B 테스트와 실험 설계의 핵심
 - ❖ 광고 효과 비교, 추천 알고리즘 비교, UI 개선 효과 측정 등 전부 평균 비교 문제로 CLT 덕분에 두 평균 차이 → 정규분포 근사 → p-value 계산 가능

Contents

1. What is Modeling?
2. Python sympy
3. Time & Space (w1)
4. Optimization (w2)
5. Linear Regression (w3) (w4)
6. Optimization of loss function (L1, L2)
7. Short cut algorithm
8. Linear Programming
9. Transportation Problem
10. Montecarlo simulation
11. Random walk simulation
12. Bootstrap
13. Central limit theorem
14. Prince Frog
15. Month Hall
16. Genetic Algorithm
17. Differential Equation

Prince Frog

82

- ❖ 당신은 앞으로 인생에서 총 N 명의 잠재적 파트너를 만날 수 있다고 가정
- ❖ 당신의 목표는 그중 가장 뛰어난 사람(X)과 매칭되는 것이며 단, 아래의 조건
 - ❖ 한 번 거절한 사람에게는 다시 돌아갈 수 없음
 - ❖ 만나는 순서는 무작위이며, 당신은 상대를 만날 때마다 점수를 매길 수 있음
- ❖ 가장 좋은 사람(X)을 선택할 확률을 최대화하려면, 몇 번째 사람을 선택해야 하나요?
 - ❖ mathematical-dating



Contents

1. What is Modeling?
2. Python sympy
3. Time & Space (w1)
4. Optimization (w2)
5. Linear Regression (w3) (w4)
6. Optimization of loss function (L1, L2)
7. Short cut algorithm
8. Linear Programming
9. Transportation Problem
10. Montecarlo simulation
11. Random walk simulation
12. Bootstrap
13. Central limit theorem
14. Prince Frog
15. Month Hall
16. Genetic Algorithm
17. Differential Equation

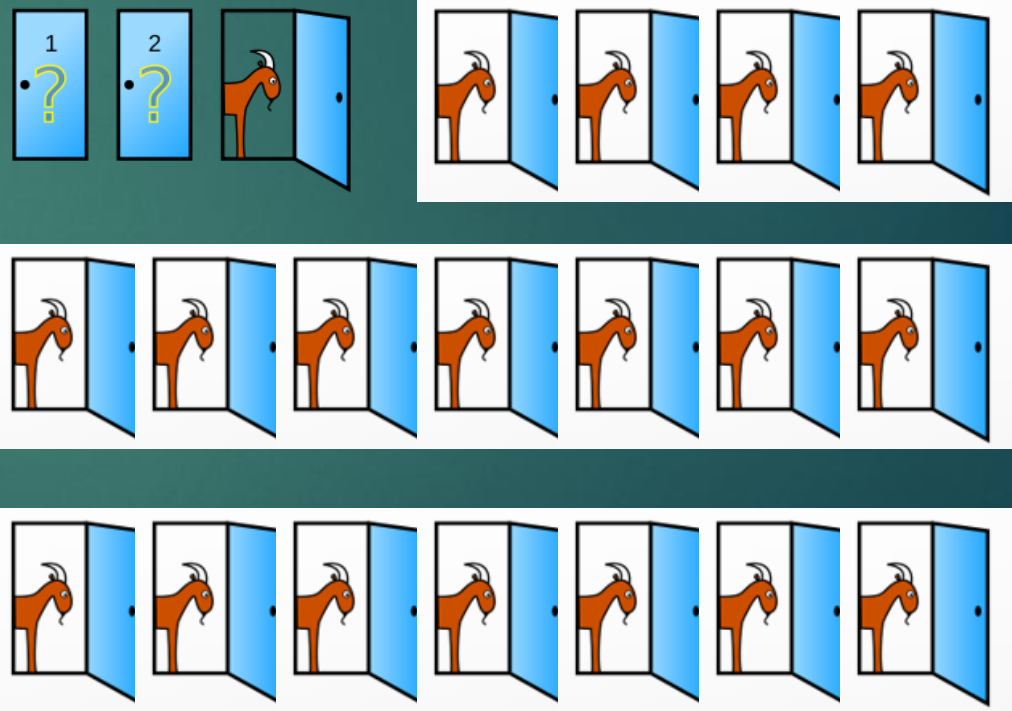
Monty Hall

84

❖ Monty Hall Problem



❖ Monty Hall Problem, 문이 10,000개 라면?



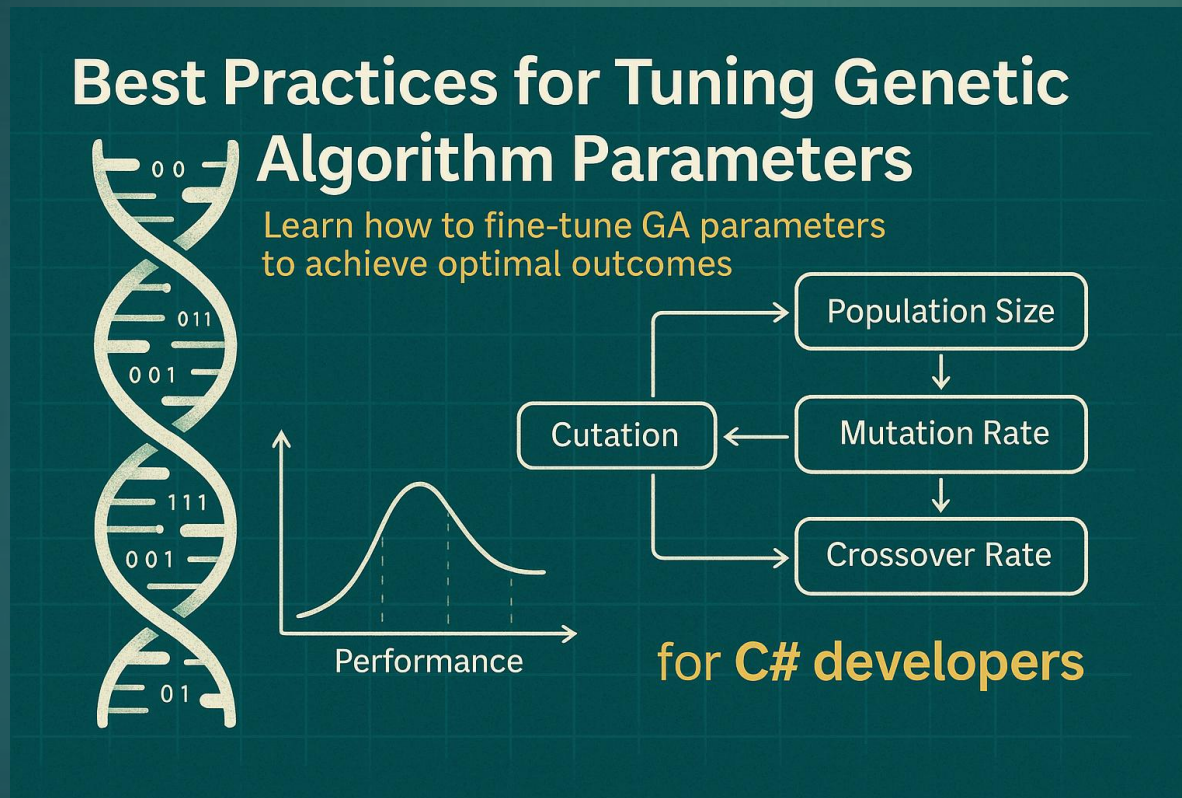
Contents

1. What is Modeling?
2. Python sympy
3. Time & Space (w1)
4. Optimization (w2)
5. Linear Regression (w3) (w4)
6. Optimization of loss function (L1, L2)
7. Short cut algorithm
8. Linear Programming
9. Transportation Problem
10. Montecarlo simulation
11. Random walk simulation
12. Bootstrap
13. Central limit theorem
14. Prince Frog
15. Month Hall
16. Genetic Algorithm
17. Differential Equation

Genetic algorithm

86

- ❖ 단순회귀식의 parameter를 유전자 알고리즘으로 구하는 파이썬 코드를 제시하라



Contents

1. What is Modeling?
2. Python sympy
3. Time & Space (w1)
4. Optimization (w2)
5. Linear Regression (w3) (w4)
6. Optimization of loss function (L1, L2)
7. Short cut algorithm
8. Linear Programming
9. Transportation Problem
10. Montecarlo simulation
11. Random walk simulation
12. Bootstrap
13. Central limit theorem
14. Prince Frog
15. Month Hall
16. Genetic Algorithm
17. Differential Equation

Differential Equation

88

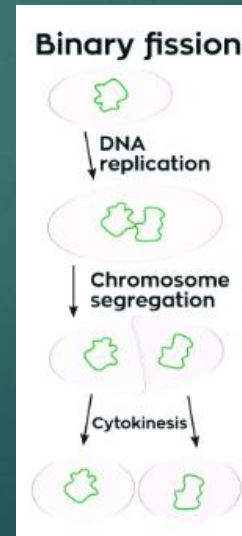
- 컴퓨터가 생물학적 또는 물리적 현상을 이해할 수 있도록 수학적으로 묘사하는 과정
 - 컴퓨터는 미분과 적분 연산에 최적화되어 있어 세포분열 현상을 미분식으로 표현하면 컴퓨터는 미래를 예측할 수 있음
 - 속도는 미분, 거리는 적분
 - 세포분열의 경우 분열 속도를 미분방정식으로 표현하고
 - 일정 시간 이후 세포는 몇 개까지 증식하는지는 적분하여 예측

Differential Equation

89

- 세포분열(Cell division) 아메바나 인체 세포는 일정 크기로 성장하면 2개로 분열하고 정상 세포는 보통 24시간 주기로 아래와 같이 분열한다.
 - 증식 속도 (Rate)
 - 속도는 “늘어난 세포의 수 / 걸린 시간”으로 정의
 - 세포가 많을수록 분열하는 개체수도 늘어나므로, 증가 속도는 현재 세포 수(X)에 비례

$$\frac{dX}{dt} = \frac{X}{24}$$



Differential Equation

90

- 세포분열(Cell division) 변화율 식을 시간에 대해 적분하면 시간에 따른 세포 수를 예측할 수 있고 이 결과를 x축은 시간 y축은 세포 수를 2차원 좌표에 표시하라
 - 1) numpy 로 오일러 방식으로 코딩하거나
 - 2) sympy 모듈로 미분방정식을 정의하고 적분

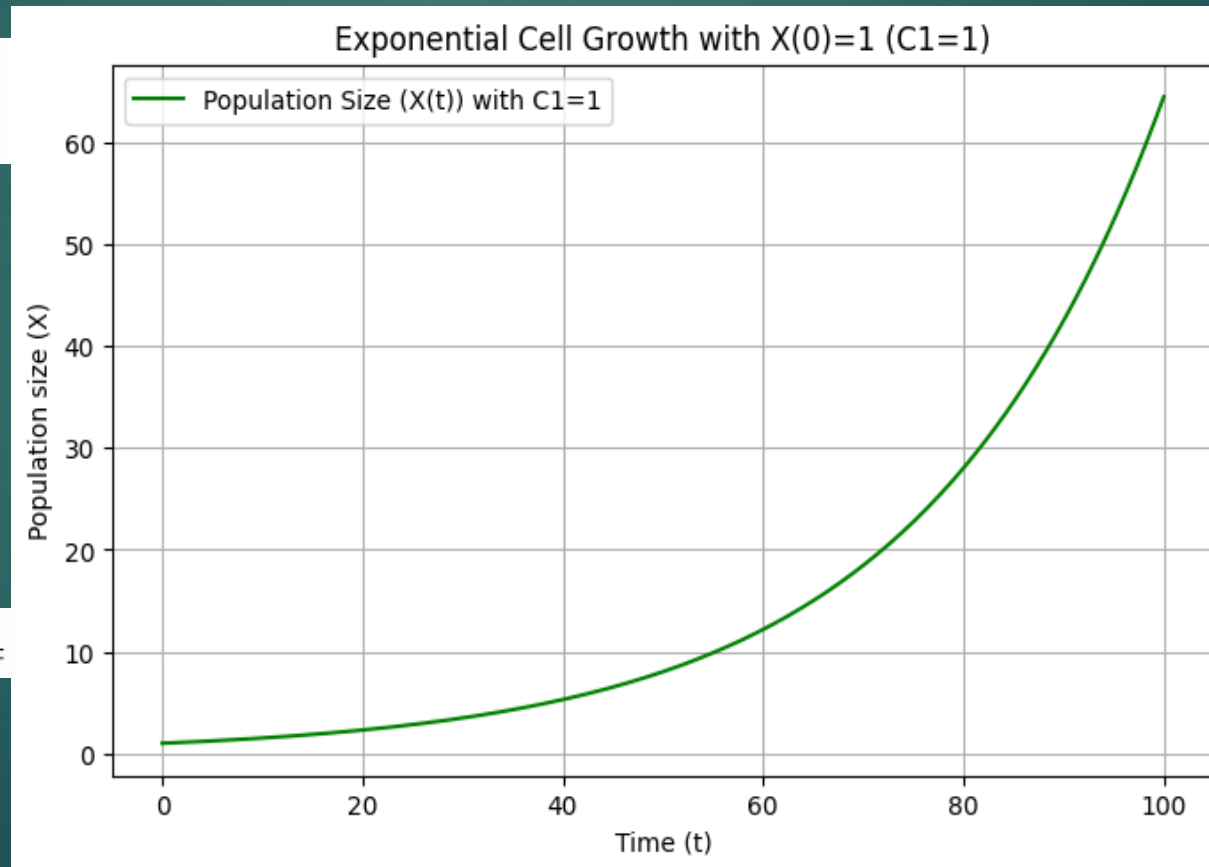
Differential Equation

91

- 24시간 이후에는 2배가 아니라 자연상수(오일러상수)인 2.718.... 배가 됨
- 세포 수가 늘어남에 따라 증가 속도가 실시간으로 가속되어 기하급수적(Exponential)으로 증가함

$$X(t) = X_0 e^{t/24}$$

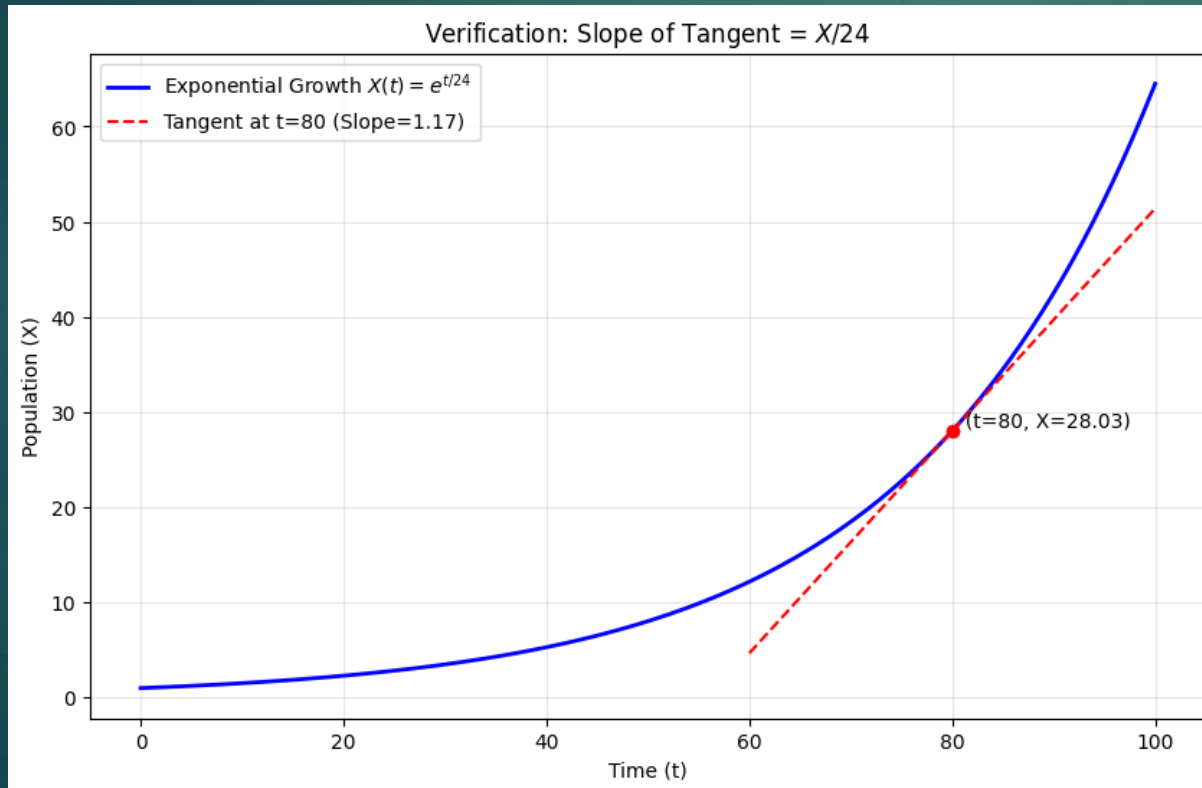
24시간 후: $X(24) =$



Differential Equation

92

- 변분법(calculus of variations) 사고
 - 미분방정식이 $t=80$ 에서는 " 너 지금 1.17의 속도로 올라가야 해! " 라는 명령어
 - 접선의 식은 그 명령어를 받아서 "그럼 일단 이 방향으로 가볼게"라고 그려본 가상의 직선 경로



이 접선을 $t=80$ 근처에서 아주 짧게 그리면 실제 세포 수 곡선과 거의 일치하게 되며 이 원리를 이용해서 컴퓨터는 복잡한 미분 방정식을 한 단계씩 직선으로 연결하며 풀어나감

1. 지점 찾기 (t_0, X_0)

- $t_0 = 80$
- $X_0 = e^{80/24} = e^{10/3} \approx 28.0316$

2. 기울기 찾기 (slope)

- $slope = \frac{X_0}{24} = \frac{28.0316}{24} \approx 1.1680$

3. 접선의 방정식 완성

공식 $y = slope \cdot (t - t_0) + X_0$ 에 위 값들을 대입

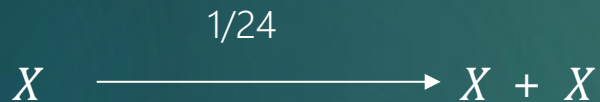
$$y = 1.1680(t - 80) + 28.0316$$

$$y = 1.1680t - 65.4084$$

Differential Equation

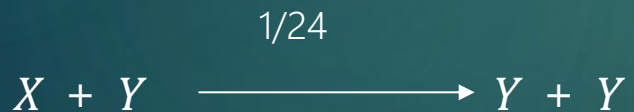
93

- X 는 정상세포 수, Y 는 감염세포 수로 가정하고 X, Y 는 다음과 같이 상호작용
 - X 는 24시간안에 세포분열 ($X \rightarrow X + X$)
 - Y 는 정상세포 X 를 24시간안에 감염시킴($X + Y \rightarrow Y + Y$)
 - X, Y 에 대한 미분방정식을 만들어라



$$\frac{dX}{dt} = \frac{X}{24} - kXY$$

- $\frac{dX}{dt} = \frac{X}{24} - kXY$: X 의 양이 시간에 따라 어떻게 변하는지.
 - $+\frac{X}{24}$: X 가 스스로 복제되는 증가 항.
 - $-kXY$: X 와 Y 가 만나 X 가 소모되는 감소 항.



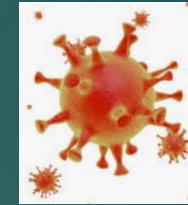
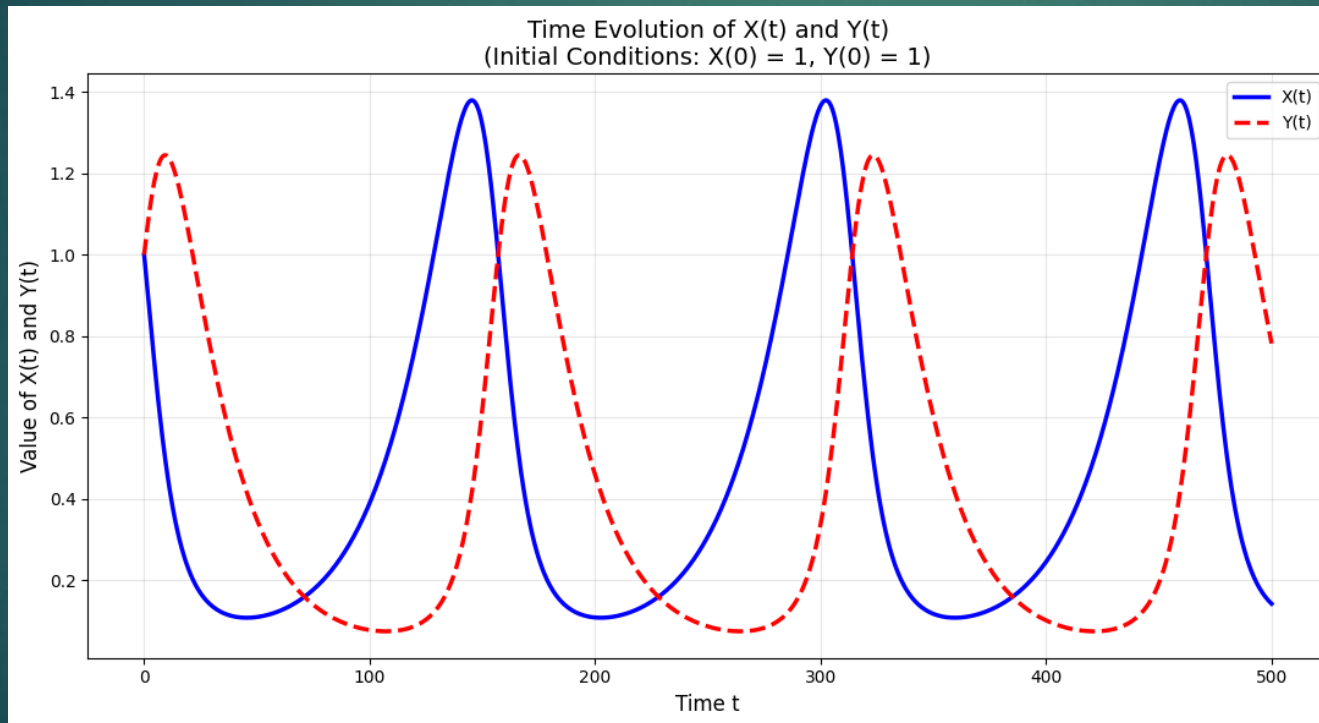
$$\frac{dY}{dt} = kXY - dY$$

- $\frac{dY}{dt} = kXY - dY$: Y 의 양이 시간에 따라 어떻게 변하는지.
 - $+kXY$: X 와 Y 가 만나 Y 가 증가하는 증가 항.
 - $-dY$: Y 가 소멸되는 감소 항.

Differential Equation

94

- X, Y 에 대한 미분방정식을 만들고, 시간(t)에 X 와 Y 의 그래프(snippet) 제시하라
 - 초기 정상세포와 감염세포의 수를 변경하면서 확인하라
 - 감염률(k)과 감염세포의 소멸율(d)가 변화하면 어떤 그래프 모양인지 확인하라

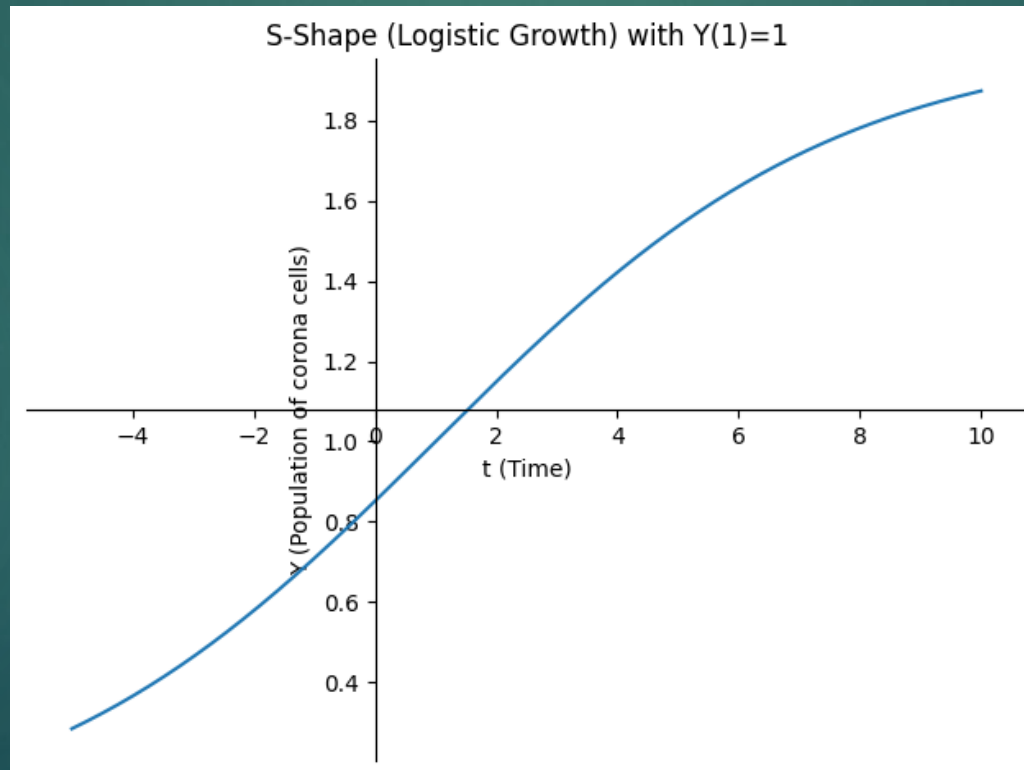


$k = 0.1$ # Reaction rate for $X + Y \rightarrow Y + Y$
 $d = 0.05$ # Decay rate of Y

Differential Equation

95

- S자 곡선(S-shape)' 형태의 전염병 확산 모델
 - 전염병 모델링에서 가장 대표적인 'S-자' 곡선은 로지스틱 성장 모델(Logistic Growth Model)
 - 이 모델은 $\frac{dx}{dt} = \frac{x}{24}$ 와 같은 단순한 미분방정식으로 표현되지 않으며, 인구 수에 비례하여 확산 속도가 달라지는 형태



Differential Equation

96

- S자 곡선(S-shape)' 형태의 전염병 확산 모델

